

Floatsom Paper

Title

FloatSOM: Topology-Flexible, Out-of-Memory (OOM)-Capable Self-Organizing Maps for Multiple Graphics Processing Units (GPUs)

Authors

Anonymous Authors

Abstract

GPU-accelerated Self-Organizing Map (SOM) implementations are among the most competitive options for large-scale SOM analysis, but growing dataset sizes increasingly challenge their practical use because workloads no longer fit cleanly within device-memory limits. We introduce FloatSOM, a GPU-oriented SOM framework for scalable training and deployment that supports multi-GPU execution, out-of-memory disk-backed streaming, and flexible topologies beyond regular lattices. We evaluate FloatSOM on 14 synthetic and real benchmark datasets together with controlled speed-scaling benchmarks, and show that improved topologies with topology-aware hyperparameter fine-tuning yield lower quantization error than current state-of-the-art SOM baselines. FloatSOM also sustains this performance at large scale with high-throughput distributed execution; in the largest benchmark, it trains a 1024-node network on 1,000,000,000 samples with 50 features in 6.16 minutes on 8 GPUs.

1. Introduction

Self-Organizing Maps (SOMs), originally introduced by Kohonen (Kohonen 1990), are an unsupervised machine learning technique that uses competitive learning to organize nodes such that they capture the topology of the data. In practice, this topology-preserving representation means SOMs are commonly used to map dataset topology, produce dimensionality-reduced visualizations, and conduct clustering at scale (Kangas et al. 1990). As dataset size and heterogeneity increase, however, the computational requirements of SOMs grow in both time and memory. Many current implementations, however, remain constrained to single-device workloads that must fit within video random-access memory (VRAM), with limited support for distributed compute, out-of-core execution, and modern GPU orchestration.

A second practical limitation concerns topology. Classical SOMs are usually trained on regular rectangular or hexagonal lattices because fixed grids make neighborhood definition, visualization, and optimization straightforward. However, these regular lattices also impose a strong geometric prior on the learned representation. Rectangular grids are especially restrictive, and even hexagonal lattices, while often preferred because they reduce directional bias (Kohonen 2013; Forest et al. 2020), still assume that the data are best represented by a fixed mesh structure. This preference for hexagonal neighborhoods is also consistent with broader tessellation literature reporting different, and often superior, behavior for hexagonal versus square neighborhood structures in spatial and quantization settings (White and Kiester 2008). Prior work on dynamic, growing, and graph-structured SOM variants reflects a long-standing recognition that fixed lattices are not always the best match for irregular data geometry (Alahakoon et al. 2000; Vasighi and Amini 2017; Kangas et

al. 1990). However, these alternatives have generally not been developed or evaluated in the high-throughput, large-sample regime targeted by modern GPU-enabled applications, and are broadly impractical for deployment at scale.

FloatSOM is designed to address these combined systems and topology limitations. Within FloatSOM, we implement distributed multi-GPU execution, out-of-memory disk-backed streaming, and scalable topology-flexible training beyond standard fixed lattices, specifically the minimum-spanning-tree (MST) and relative-neighbourhood-graph (RNG) topologies. We also evaluate multiple sampling strategies as a route to further computational acceleration, and derive fine-tuned hyperparameter configurations across diverse datasets and recommended operating regimes. FloatSOM is suitable for both high-performance computing (HPC) operation, and also operation on consumer-grade desktop GPUs.

2. Related Work

Related work on practical SOM deployment spans software implementations, sampling-efficient training, topology design, and model selection.

2.1 Open-Source SOM Implementations and Systems

Open-source SOM implementations span lightweight libraries to more performance-oriented systems. MiniSom is a commonly employed compact Python SOM implementation. It implements a CPU-resident online training procedure in which weight updates are applied sequentially at the sample level, as per the classical SOM algorithm (Vettigli 2018). XPySOM is another Python based SOM implementation that uses the newer ‘Batch’ training regime (Mancini et al. 2020). In batch training, each iteration evaluates the presented samples against the map, accumulates neighborhood-weighted update statistics across that set, and then applies a coordinated prototype update. In most modern practical settings, this BatchSOM regime has become the dominant approach because it offers superior stability, speed, and output quality relative to online sample-wise updates, while also aligning naturally with vectorized linear algebra and GPU execution (Kohonen 2013; Mancini et al. 2020). Finally, in terms of distributed compute, the Somoclu and GigaSOM are perhaps represent the most mature implementations. Somoclu emphasizes parallel large-scale training through a C++ core (Witek et al. 2017), whereas GigaSOM.jl provides a distributed implementation in the Julia language, targeted at very large cytometry workloads (Kratochvíl et al. 2020).

Taken together, these implementations represent the frontier of scalable and open SOM implementations. Among the openly available systems considered here, the highest reported single-machine throughput implementation is XPySOM, by virtue of its efficient GPU backed operations (Mancini et al. 2020). GigaSOM.jl, in turn, represents the most strongly scaled openly available distributed implementation in this area, including the processing of a 1,167,129,317-cell dataset on a 256-core distributed CPU cluster in slightly under 25 minutes (Kratochvíl et al. 2020). Nevertheless, GigaSOM’s implementation within the comparatively smaller Julia ecosystem nevertheless may limit interoperability with the broader Python-centered scientific software stack and with some established HPC workflows. Additionally, it does not have GPU support to further accelerate the algorithm. Accordingly, an important systems gap remains: an openly usable distributed GPU SOM implementation that combines the throughput of GPU batch pathways with execution beyond single-device in-VRAM constraints.

2.2 Sampling Methodologies for SOM Training

Classical online and batch SOM training schemes traditionally sample every data-point in every training iteration (Kohonen 2001; Liu et al. 2023). Consequently, a readily appreciable method of further optimising SOMs for increased speed and scalability would be the employment of a sub-

sampling regime - effectively reducing the amount of data required to train the final SOM. Note that this sampling question is separate from the online-versus-batch distinction: a SOM can use full or sampled data under either update regime, whereas online and batch refer to how updates are accumulated and applied.

To date, numerous subsampling strategies have been proposed. Beyond naive random sampling, adaptive sampling methods such as hierarchical dynamic subset selection SOM (HDSSSOM), which concentrates computation on difficult or stale regions of the data (Wetmore et al. 2005), and SOM-based adaptive sampling for design-space exploration, which iteratively guides new evaluations toward promising or underexplored regions, have also been introduced (Ito et al. 2016). Hence, the literature offers several plausible sampling strategies, but systematically benchmarked and openly maintained implementations that compare full-data, random, and guided sampling within the same modern high-performance SOM workflow remain limited.

2.3 Regular Grids and Alternative Topologies

Most practical SOM implementations retain regular rectangular or hexagonal lattices because they simplify neighborhood indexing, visualization, and vectorized updates (Kohonen 1990; Kohonen 2013). Hexagonal grids are often preferred in the literature because their neighborhood geometry is more isotropic, tends to reduce directional bias, and produces more accurate results relative to rectangular grids (White and Kiester 2008; Kohonen 2013; Forest et al. 2020).

At the same time, the SOM literature has explored alternatives to fixed lattices, considering that realistic data distributions often do not map cleanly onto fixed lattices. Consequently, this has yielded attempts to rectify this shortcoming by neighborhood function modification (Aoki and Aoyagi 2007). Other attempts include the development of dynamic maps which change their configuration, node number, or connections throughout the training cycle, such as in DBGSOM and AMSOM (Vasighi and Amini 2017; Spanakis and Weiss 2016). Graph-structured neighborhoods have also been proposed, including minimum spanning tree formulations in early SOM work (Kangas et al. 1990) and later smaller-scale MST-based analyses (Jang et al. 2009). Relative Neighborhood Graphs (RNGs) provide another sparse geometry-driven topology that can be viewed as a middle ground between MST-style graph neighborhoods and regular lattices such as hexagonal grids, but to our knowledge this possibility has not been meaningfully examined in openly available SOM toolchains. Nevertheless, both implementations are not publically available.

Together, these evince a need for topologically-flexible SOM proposals in high-throughput SOM systems. Irregular neighborhoods are harder to construct, refresh, and query efficiently during training than fixed lattices, so openly usable implementations that make non-lattice topologies practical under modern large-scale execution remain elusive.

2.4 Hyperparameter Optimization and Fair Comparison

SOM performance depends strongly on choices such as map size, initialization, learning-rate schedule, and neighborhood schedule. Prior work shows that these choices can materially affect observed performance and even the apparent advantage of one variant over another (Akinduko et al. 2016; Forest et al. 2020). This makes comparisons based only on untuned defaults difficult to interpret.

More generally, modern machine-learning workflows increasingly rely on automated hyperparameter optimization rather than manual tuning alone. Frameworks such as Optuna provide bounded search over large hyperparameter spaces and can support multi-objective optimization, allowing parameter settings to be selected with respect to several benchmark criteria simultaneously rather than collapsed into a single score (Akiba et al. 2019).

Currently, the literature provides strong components in isolation: accessible SOM libraries, accelerated batch implementations, classical and guided sampling schemes, and multiple alternatives to standard grid neighborhoods. What remains limited is an openly usable workflow that combines these pieces in one benchmarkable setting: support for both regular and irregular topologies, direct comparison of full, random, and guided sampling, and execution beyond a narrow single-device in-memory regime. Similarly, in combining these optimal configurations, empirically derived default hyperparameters are also essential for optimal hyperparameters. This evidence based derivation and analysis of performance impact is, to date, an unexplored area.

3. Methods

A standard Self-Organizing Map (SOM) can be viewed as a small set of interacting components: the selection of training samples at each iteration, the definition of neighbourhood relations between map units, the batch training step that updates the prototypes from those sampled data, and the compute framework used to execute those operations. In FloatSOM, the core batch training formulation is kept close to the standard SOM, while the main methodological changes are introduced in the sampling, topology, and execution components. Specifically, the sample component determines which data are presented at each iteration (`random`, `full`, or `HDSSOM` in FloatSOM), the topology component defines the neighbourhood relations between map units (rectangular, hexagonal, MST, or RNG), and these two choices feed into the standard batch training procedure. The compute framework then determines how that same training procedure is executed in practice, ranging from local GPU execution to single-node multi-GPU and multi-node GPU settings. Hyperparameter optimization is treated as an additional methodological layer applied across these configurations. Figure 0 provides a schematic overview of these components and their FloatSOM options; the following subsections then describe each component in turn.

Regardless of the selected topology, FloatSOM uses the same prototype initialization options. Initialization determines only the starting prototype values; neighbourhood relations are applied afterward according to the selected topology, keeping the initial state comparable across regular-lattice and graph-based runs.

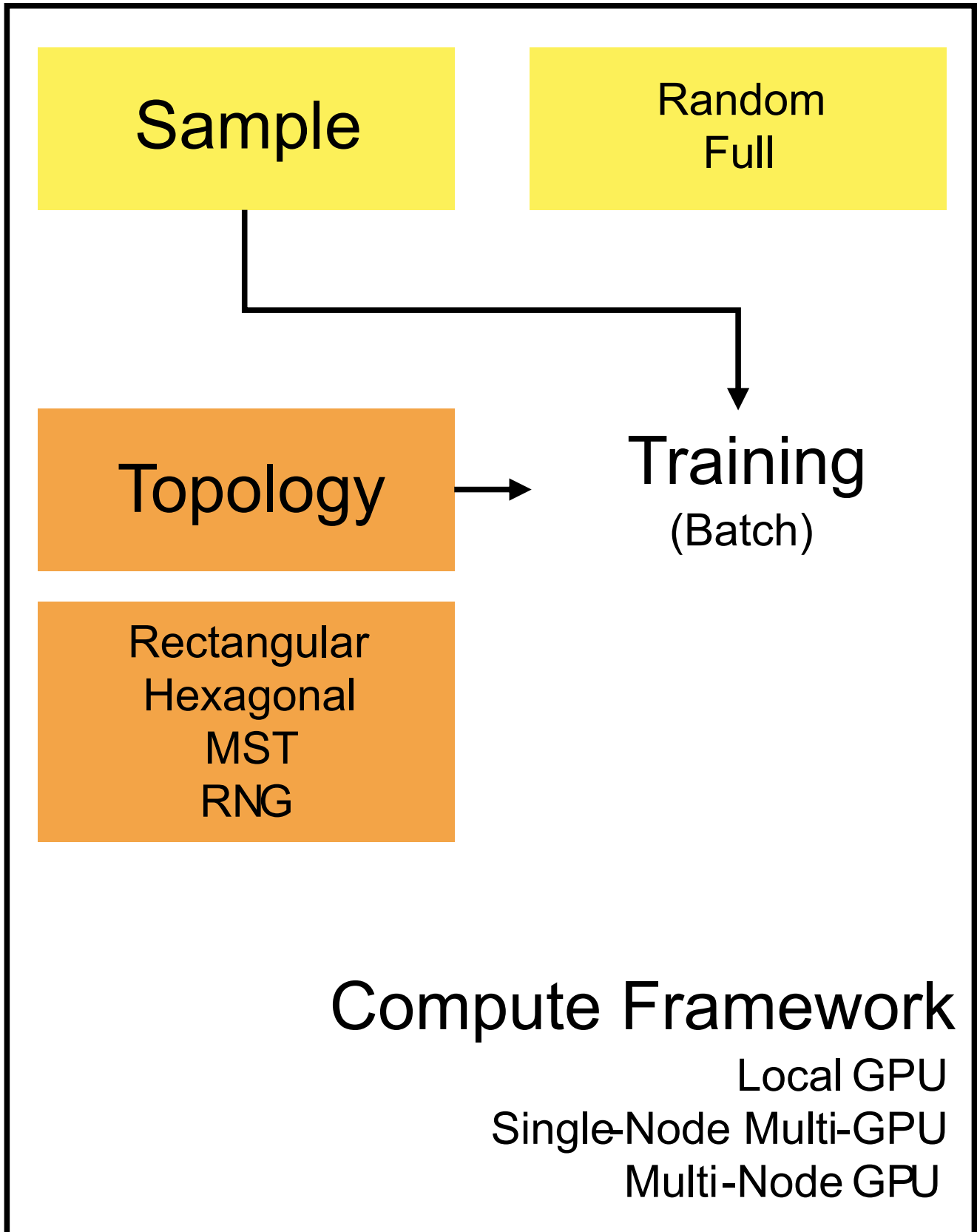


Figure 0

Figure 0. Schematic overview of the FloatSOM methods framing used in this manuscript. Sample selection and topology definition have configurable components that feed into the standard batch SOM training step, while the compute framework determines how that same training procedure is executed in practice. The options shown here summarize the FloatSOM configurations discussed in the following subsections.

3.1 Sampling Selector Mathematics

This section formalizes the sampling policies evaluated in this work.

Let the full dataset be $X = \{x_i\}_{i=1}^N$. Let m denote the number of samples presented to the SOM in a given training iteration, or the ‘sampling budget’. This budget is either fixed directly or determined as a proportion ρ of the dataset:

$$m = \begin{cases} m_0, & \text{if a fixed budget is specified,} \\ \max(1, \lfloor N\rho \rfloor), & \text{if a dataset proportion is used.} \end{cases}$$

For index sets $\mathcal{J}_t^{(s)}$ at iteration t , the implemented full and random selectors are:

$$\mathcal{J}_t^{\text{full}} = \{1, \dots, N\}, \quad \mathcal{J}_t^{\text{random}} \sim \text{Unif}(\{I \subseteq \{1, \dots, N\} : |I| = m\}), \quad m < N.$$

If $m \geq N$, random returns the full dataset (no subsampling). The selected training batch is $X_t^{(s)} = \{x_i : i \in \mathcal{J}_t^{(s)}\}$.

For random subsampling, the per-iteration objective is aligned in expectation with full-data risk:

$$\mathbb{E}_{\mathcal{J}_t^{\text{random}}} \left[\frac{1}{m} \sum_{i \in \mathcal{J}_t^{\text{random}}} \ell(x_i; W) \right] = \frac{1}{N} \sum_{i=1}^N \ell(x_i; W).$$

For hierarchical dynamic subset selection SOM (HDSSSOM) (Wetmore et al. 2005), the core algorithm is kept from the original publication and re-implemented here to be multi-GPU compatible. Briefly, HDSSSOM is an adaptive sampling strategy that aims to focus computation on informative regions of the dataset by preferentially revisiting samples that are difficult, under-trained, or stale, while still preserving exploration across training.

3.2 Topology Definition

We next define how neighbourhood structure is assigned in FloatSOM across regular-lattice and graph-based configurations.

All topologies support the same node placement initialization options, and are initialised in an identical manner. Post initialization, neighbourhood relations are defined by the selected topology. For regular-lattice baselines, we support both grid and hexagonal layouts, but we treat hexagonal as the standard topology reference in this manuscript based on prior SOM guidance. For MST and RNG, neighbourhood structure is instead derived from the current prototype geometry using the below methodologies.

3.2.1 MST Topology Implementation

MST topology replaces fixed lattice neighborhood distance with graph shortest-path distance on a minimum spanning tree built from current prototypes. For P prototype nodes in feature dimension d , the pairwise prototype matrix is formed with the standard squared-distance Gram identity, which avoids 3D broadcast tensors and preserves $O(P^2 d)$ dense linear-algebra structure.

After distance construction, we build a minimum spanning tree over the prototypes and use shortest-path distances on that tree to evaluate the Gaussian neighborhood influence during learning (Kruskal 1956). Topology-derived influence matrices are cached and refreshed at fixed or progress-adaptive

intervals.

Under this refresh policy, the topology path is simple: if the current iteration does not trigger recomputation, the previous graph state and cached influences are reused; otherwise, pairwise prototype distances are rebuilt on GPU, the MST is recomputed on CPU, graph distances are refreshed in chunked GPU fashion, and the influence cache is rebuilt for the deduplicated active radii.

Input: prototypes W_t , iteration t , topology-refresh policy

Output: topology state (E_t , g_t , cached influences)

```

1: Query the refresh policy for the current iteration
2: if no topology refresh is due then
3:     return previous topology state
4: end if
5: Compute pairwise squared prototype distances on GPU
6: Transfer distances to CPU and run Kruskal to obtain MST edges  $E_t$ 
7: Build adjacency from  $E_t$ 
8: Compute all-pairs graph distances  $g_t$  with chunked GPU Floyd-Warshall
9: Deduplicate active radii and rebuild/update cached influence maps
10: Commit  $E_t$ ,  $g_t$ , and cache state
11: return topology state

```

Algorithm 3. Dynamic MST topology update with refresh-triggered recomputation and cached influence reuse.

3.2.2 RNG Topology Implementation

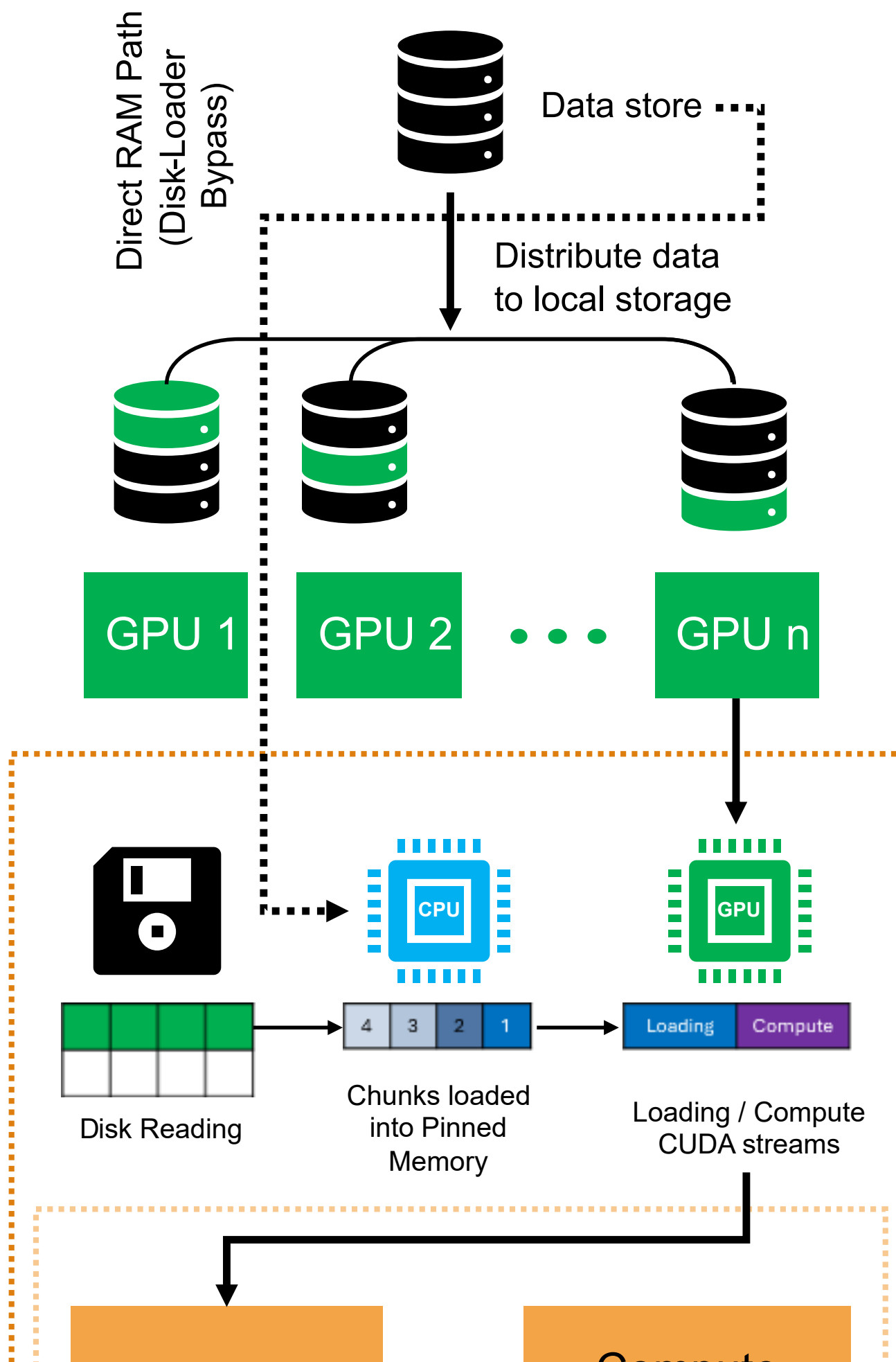
RNG is our second topology contribution. Our RNG topology constructs a Relative Neighborhood Graph over current prototype distances using the standard RNG criterion (Toussaint 1980), and then reuses the MST infrastructure for shortest-path precomputation, radius-deduplicated influence caching, and dynamic update scheduling. In this sense, RNG serves here as an alternative sparse topology that sits between MST and regular lattice baselines such as the hexagonal grid.

Relative Neighborhood Graphs are less constrained than MSTs as they are not restricted to a single spanning-tree backbone with exactly one route between connected prototypes. Instead, when local geometric evidence supports multiple neighborhood relations, RNG can retain those connections rather than forcing the structure through only one edge choice per region. Consequently, we hypothesise that this added flexibility will permit more faithful recovery of real data-local connections and, as a consequence, a superior topology relative to MST.

We implement the RNG topology by evaluating candidate elimination in chunks to control memory pressure while preserving the direct strict blocker test. No post-hoc connectivity repair is applied after edge extraction; the topology is defined entirely by the canonical RNG criterion.

3.3 Multi-GPU + OOM Methodology and Implementation

This section covers how we distribute computation across GPUs, how data are streamed for large workloads, and how memory safeguards preserve progress under high-pressure regimes.



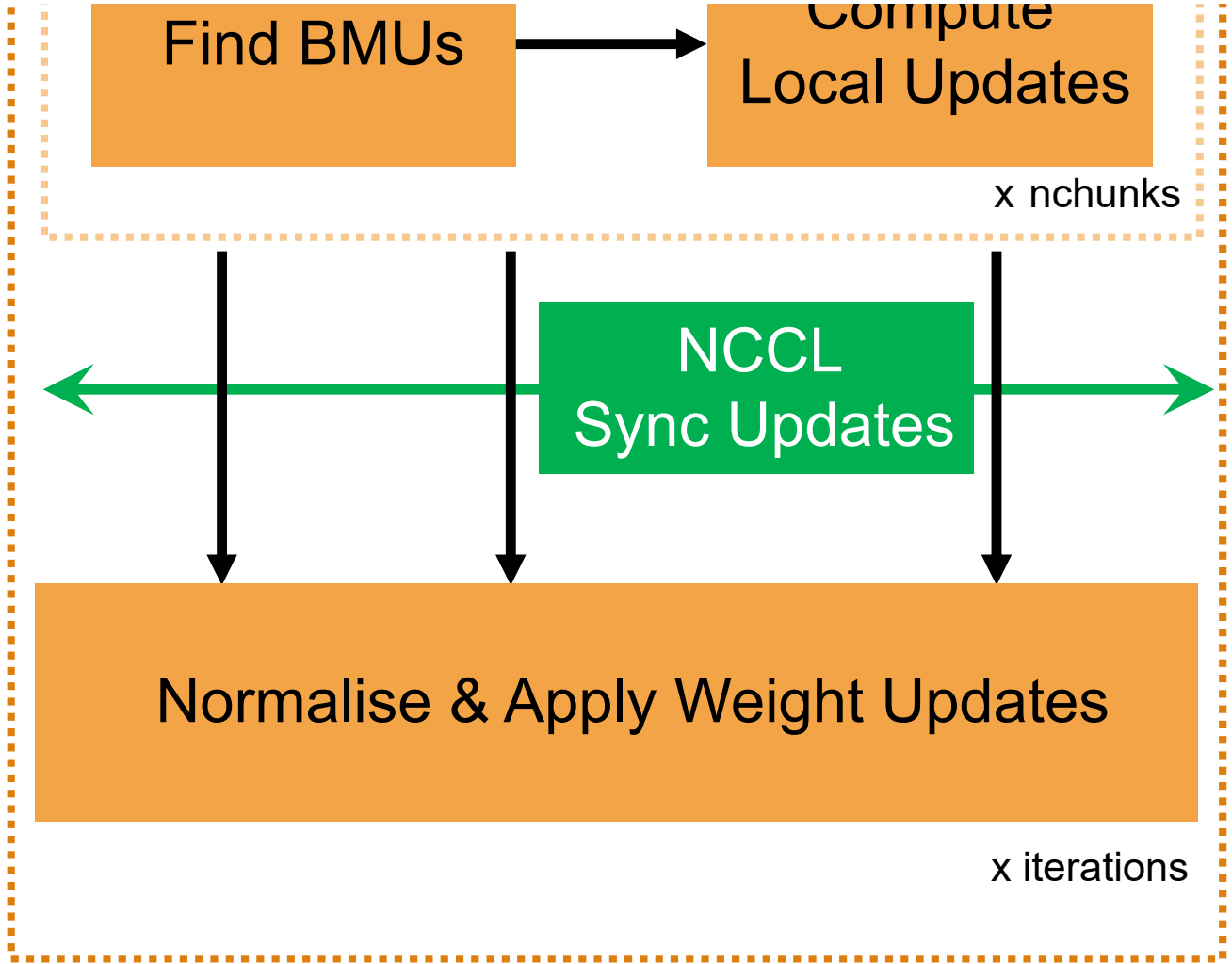


Figure 1

Figure 1. Multi-GPU data-loader and NVIDIA Collective Communications Library (NCCL) synchronization schematic. Solid disk-backed path: data are distributed from shared storage to worker-local shards on node-local storage, read in worker-local chunks (`n_chunks`) into pinned host memory, transferred to GPU while loading overlaps with compute, processed as chunked local BMU/update steps, synchronized by NCCL all-reduce, and normalized into one weight update per iteration. Dotted path: random-access memory (RAM) mode, where data are sharded directly into each worker's GPU-local CPU RAM and follow the same pinned-memory-to-GPU path without disk reads; this path is typically faster when data are already memory-resident because disk-read overhead is removed.

3.3.1 General Multi-GPU Logic

Distributed execution uses Ray actors with one GPU per worker and NCCL collectives for synchronous aggregation (Moritz et al. 2018). For every iteration, each worker processes its assigned shard locally. As illustrated in Fig. 1, that shard is processed within the worker as `n_chunks`; Eqs. (4)-(5) are written at the shard level, but in implementation the worker-local accumulators are built incrementally across those chunks before synchronization. Finally, upon worker-local computations being completed, influences are accumulated across workers via NCCL synchronisation.

For worker $g \in \{1, \dots, G\}$, let $X_{t,g}^{(s)}$ denote the worker-local shard of the selected iteration batch $X_t^{(s)}$. Let j index prototype nodes, let $w_j^{(t)}$ denote the prototype vector of node j at iteration t , let $b(x)$ denote the best-matching unit (BMU) of sample x under the current weights, let $h_{j,b(x)}^{(t)}$ denote the

iteration- t neighborhood influence between node j and the BMU of x , and let η_t denote the learning rate at iteration t . The local accumulators are:

$$U_j^{(g)} = \sum_{x \in X_{t,g}^{(s)}} \eta_t h_{j,b(x)}^{(t)} (x - w_j^{(t)}),$$

$$H_j^{(g)} = \sum_{x \in X_{t,g}^{(s)}} h_{j,b(x)}^{(t)}.$$

Global synchronized accumulators are:

$$U_j = \sum_{g=1}^G U_j^{(g)},$$

$$H_j = \sum_{g=1}^G H_j^{(g)}.$$

In implementation, $U_j^{(g)}$ and $H_j^{(g)}$ are accumulated across the worker's `n_chunks` and synchronized once per iteration. Normalization and momentum are then applied with globally consistent denominators.

3.3.2 Multi-GPU Implementation Details

As shown in Fig. 1, each worker uses a chunked loading path from CPU memory to GPU memory. In streaming mode, data are distributed to worker-local disk shards and then read chunk-by-chunk into pinned host memory before transfer to GPU. In RAM mode, data are pre-sharded directly into each worker's GPU-local CPU RAM and fed into the same pinned-memory path, bypassing disk reading. In both cases, the worker processes its assigned shard as `n_chunks` rather than materializing the full shard on device.

The loader keeps only a small number of upcoming chunks in memory and fetches the next chunk scheduled after the current one. We operate multiple CUDA streams, so that, in steady state, one chunk can be under GPU compute while the next chunk is being transferred and another host buffer is being prepared. For each chunk, the worker performs BMU search and accumulates local update and influence tensors.

After all required data for the current iteration have been processed on each worker, NCCL performs a synchronous all-reduce over the worker-local accumulators, allowing for worker-local weight normalisation and updating. Consequently, weights remain resident on worker GPUs across iterations, instead the driver exchanges only lightweight metadata rather than full weight tensors.

3.3.3 OOM-Capable Topology Updates

Additional larger-than memory support for topology updates is provided through topological chunking. Topological chunking applies the same idea to topology-side computations within each worker. When graph-distance or influence structures would otherwise exceed a worker's memory budget, those computations are tiled and evaluated in bounded pieces rather than materialized at once. This topology-side chunking is separate from the data path shown in Fig. 1, but it follows the same per-worker bounded-memory execution rule.

3.3.4 XPySOM vs FloatSOM batch comparison

FloatSOM offers the option for matching to XPySOM, to produce identical results when configured in the 'XPySOM' equivalence mode.

3.4 Multi-Objective Hyperparameter Optimization

We use Optuna as an automated multi-objective hyperparameter optimization framework to derive near-optimal performance and corresponding hyperparameters for each FloatSOM configuration under a given sampling, topology, and processing combination (Akiba et al. 2019). In this setting, multi-objective optimization means searching for parameter configurations that jointly balance the selected benchmark objectives rather than optimizing a single scalar criterion.

4. Experimental Setup

We use two benchmark protocols: an Optuna quality benchmark and a speed-scaling benchmark. The first evaluates algorithmic quality and tuned attainable performance, and the second evaluates runtime and distributed scaling behavior. All production Optuna and speed benchmarks reported in this manuscript were executed on Gadi at the National Computational Infrastructure (NCI), Australia, on gpuvolta nodes (*HPC Systems NCI*, n.d.), using a consistent multi-GPU environment across runs.

4.1 Optuna benchmark protocol

We use Optuna-based multi-objective optimisation to determine the best attainable performance and corresponding hyperparameters for each sampling (full and random) and topology (hexagonal, MST, and RNG) combination. Presently, we chose to optimise for both QE_T and QE_H . Hyperparameter search is run with configuration constraints that depend on the selected algorithmic variant, so comparisons remain consistent across datasets while allowing variant-appropriate tuning spaces. A separate focused HDSSSOM sampling pilot is reported later in Section 5.2; unlike the main full-versus-random benchmark, that pilot uses a restricted run envelope summarized in Supplementary Table S2.

Concretely, each dataset-topology-sampling configuration is optimized for 200 Optuna trials and replicated across 10 seeds to ensure robust results. Trials are executed sequentially within each run, with the current optimum updated after each completed trial. Batch mode is fixed to full-batch training in this protocol for the benchmark results reported in this manuscript. Operationally, this Optuna campaign uses the standard in-memory batch path rather than the Ray-distributed execution stack due to dataset size not requiring Ray. This corresponds to:

$$14_{\text{Datasets}} \times 10_{\text{Seeds}} \times 3_{\text{Topologies}} \times 2_{\text{SamplingMethods}} \times 200_{\text{Trials}} = 168,000 \text{ Runs}$$

Sampling comparisons in Section 5.2 use the hexagonal subset of this campaign for full-vs-random paired analyses, and include a focused HDSSSOM pilot under its own restricted configuration. Topology comparisons in Sections 5.3-5.4 use full-sampling runs across hexagonal, MST, and RNG and include all available full-sampling datasets in this subset (no dataset-size exclusion for Figures 5-6). Unless explicitly stated otherwise, the remaining analyses reported in this manuscript use full sampling with full-batch training.

4.1.1 Optuna benchmark datasets and preprocessing

The Optuna benchmark uses a mixture of synthetic and real datasets from scikit-learn (Pedregosa et al. 2011) to expose the algorithm to a broad range of challenges and verify stable behavior across distinct data regimes. Synthetic datasets include: *swiss_roll*, *moons*, *circles*, *blobs*, *s_curve*, and real datasets include: *breast_cancer*, *wine*, *iris*, *digits*, *olivetti_faces*, *diabetes*, *california_housing*, *covertype*, *kddcup99*. Synthetic datasets are generated in accordance with the random seed, while real-world datasets are loaded directly from sklearn. Across the Optuna protocol, inputs are standardized feature-wise to zero mean and unit variance using `StandardScaler` before deterministic seeded permutation and a fixed 70/30 train-holdout split. We retain both train and

holdout partitions because they capture two different practical questions: how well the SOM represents the observed training population, and how well that same trained map transfers to previously unseen samples.

4.1.2 Optuna quality metrics

Our primary quality metric is Quantization Error (QE), computed in the standard way with GPU distance kernels. We report both train and holdout QE , denoted QE_T and QE_H , respectively. Here, QE_T captures use cases where the full observed population is available and the map is intended to represent that same population, while QE_H captures generalization settings where the trained SOM is projected onto previously unseen samples.

Balanced QE , denoted QE_B , is defined as the mean of QE_T and QE_H . QE_B is therefore a composite endpoint that weights representation fidelity (train) and transfer-to-unseen-data fidelity (holdout) equally. Unless stated otherwise, we report raw (non-normalized) QE_B . Again, note that in the Optuna runs, QE_T and QE_H are optimized jointly as a two-objective vector, with QE_B only calculated *post hoc*.

4.2 Speed-scaling benchmark protocol

The speed-scaling benchmark evaluates runtime and distributed scaling behavior rather than train-holdout generalization. In the main scaling experiments, sampling is fixed to full, the entire generated sample set is used for training, and no holdout split is applied. A dedicated 1-GPU batch-mode random-versus-full comparison is additionally included to isolate sampling-specific runtime effects independently of the multi-GPU scaling runs. The objective of the speed-scaling benchmark is to characterize the speed behavior of different compute and algorithm configurations.

Speed scaling is evaluated with harmonized runs across $G \in \{1, 2, 4, 8\}$ GPUs under a fixed sample-scaling protocol. Runtime summaries are computed from repeated executions per configuration and reported as both absolute training time and efficiency ratios relative to a 1-GPU comparison. These scaling and multi-GPU/OOM-capable runs use the Ray-orchestrated distributed execution layer built on top of the standard FloatSOM training path (Moritz et al. 2018). Accordingly, the scaling figures in Sections 6.1-6.2 and the runtime/scaling comparison reported later against XPySOM should be interpreted as distributed-execution results rather than the in-memory Optuna path.

For scaling-efficiency calculations, when a single-GPU baseline was missing at a given axis value due to timeout, we estimated that baseline by local linear extrapolation from the last available 1-GPU point on the same curve. That is, runtime was assumed to scale proportionally with the axis variable for the extrapolation step (for example, doubling sample count or doubling dimensionality doubles the estimated 1-GPU runtime). This assumption was used only to construct missing 1-GPU baselines for speedup/efficiency reporting and did not alter the observed multi-GPU runtime traces.

Topology-speed comparisons include hexagonal, MST, and RNG, with harmonized workload settings so ratios isolate topology-associated runtime effects.

4.2.1 Scaling benchmark datasets

The speed benchmark uses synthetic random matrices with uniform values in $[0, 1]$ and evaluates scaling under controlled sample-size sweeps. No test-holdout split was used here because the objective is runtime rather than generalization, so all generated data are used for training.

Compute resources are scaled with GPU count while keeping the software environment and benchmark procedure consistent across runs. For scaling benchmarks, each run was terminated at a wall-clock timeout of 30 minutes (timeout). Per-GPU resource allocation was fixed at 12 CPU cores,

90 GB system RAM, and a NVIDIA V100 with 32 GB VRAM, with one full node composing of 4 GPUs and their associated CPUs, with 400GB of local associated disk storage.

4.3 Statistical analysis

All Optuna comparisons were paired because the experimental design contains substantial between-run heterogeneity arising from dataset, seed, and split structure. Pairing therefore allows each comparison to be evaluated within a matched experimental context.

Pairs were defined within matched dataset, seed, and split units, while holding all non-target descriptors fixed and varying only the factor under study; batch mode was included as an additional matching constraint when relevant. Within each matched unit, trials were ranked by the target metric, the top five were retained, and each condition was summarized by the median of those retained trials. Equivalently, our primary summary uses a top- k rule with $k = 5$, where k denotes the number of best-ranked trials retained per matched unit before taking the within-condition median. This summary is intended to estimate near-optimal attainable performance under the common Optuna budget rather than average behavior across all explored hyperparameter settings. Paired effects were then computed as simple condition differences, with negative values favoring the first condition for lower-is-better metrics.

Dataset-level paired effects are visualized as forest plots. For each dataset row, the point estimate is the sample mean of the paired effects contributing to that row, and the horizontal whiskers show the corresponding 95% confidence interval from the same two-sided paired one-sample t -test applied to those paired effects. We use this paired t -test because the estimand in the forest plots is the mean paired effect and each dataset contributes multiple matched pairs under the executed design.

Global aggregate summaries use the same paired t -test framework. The single global row is tested with a two-sided paired one-sample t -test on the paired effects contributing to that aggregate, together with the corresponding 95% confidence interval around the mean paired effect. The same paired t -test summary is used for top- k sensitivity analyses. In this manuscript, wording such as “no detected difference” means failure to reject at the stated threshold under this paired t -test pipeline; it is not a formal equivalence claim.

4.4 Tuned Configuration-versus-untuned Reference Analysis

To quantify parameter-tuning benefit, we performed an explicit paired analysis between a tuned configuration and an untuned reference on seed-preserving Optuna exports. Pairing was defined within matched dataset, seed, dataset split, topology, and sampling mode. Within each matched unit, the untuned default XPySOM values was taken as the reference, while the tuned comparator was the best trial by the target metric (QE_T , QE_H , or calculated QE_B).

Dataset-wise tuned-configuration-versus-untuned-reference summaries use the same forest-plot convention, with point estimates given by the sample mean paired effect and horizontal whiskers showing the corresponding 95% confidence interval from the two-sided paired one-sample t -test. The global overall summary pools all matched tuned-configuration/untuned-reference pairs across datasets and applies the same paired t -test and confidence-interval construction to that pooled paired set.

4.5 Hyperparameter stability and dataset-type stratification

An additional aim was to determine whether the topology and sampling strategies considered here differ in hyperparameter stability. In this context, stability refers to the extent to which similar high-performing hyperparameter settings are replicated across repeated runs and matched experimental

conditions, such that a single parameter configuration can be applied with confidence across a broad range of scenarios while still yielding good performance.

Hyperparameter stability was evaluated from the top-ranked tuned Optuna trial selected separately for each matched dataset, seed, topology, processing mode, sampling mode, and optional split. Stability was then quantified as the variation in those selected hyperparameter values across seeds within each topology. Greater stability is practically important because it reduces the need for user hyperparameter retuning and increases confidence in the robustness of the resulting map quality.

For numeric parameters, stability was measured across within-topology seed pairs using the relative difference:

$$\frac{|a - b|}{\max(|a|, |b|, \varepsilon)}$$

Where a and b are the values of a given numeric parameter for the two compared seeds, and with a small $\varepsilon = 10^{-12}$. These relative differences were then averaged over the compared numeric parameters; lower values indicate higher stability. For categorical parameters, stability was measured as the mean mismatch rate across the same seed pairs and the compared categorical parameters, again with lower values indicating higher stability. Topology comparisons report both per-parameter stability scores and an equal-weight overall summary.

For dataset-type stratification, we use the same synthetic/real group definitions introduced in Section 4.1.1. This enables direct synthetic-versus-non-synthetic interpretation for both tuning and topology-stability outcomes.

5. Results

5.1 XPySOM calibration (Equivalence)

[@Givanna] I see where you are coming from, but I think it's important to establish early on to the reader that they can have confidence in the Hexagonal FloatSOM baseline as being comparable to the current methodology. Therefore, we can validly compare everything else to come, what do you think?

Under matched-configuration XPySOM-versus-FloatSOM calibration on hexagonal QE (Fig. S3), the two implementations are numerically equivalent up to expected floating-point accumulation-order effects (e.g., backend/kernel reduction order and host-device execution details), not algorithmic-update differences. Using the paired-testing pipeline defined in Section 4.3, we find that paired QE differences remain small across datasets and splits in the hexagonal calibration (Supplementary Table S5). Accordingly, we treat FloatSOM batch as a valid proxy for XPySOM in the Optuna benchmarks that follow.

We do not include MiniSom as a full benchmark baseline in the remaining experiments. This scope choice reflects both prior literature already supporting the expected online-versus-batch behavior context for this implementation class and the practical runtime cost of MiniSom at the scales targeted here. Under iteration-matched training semantics, the standard benchmark configuration used in this manuscript, with 10^7 samples, 50 dimensions, a 32×32 grid, and 10 iterations, requires MiniSom more than 12 hours to complete, making it impractical for the broader comparative campaign. Within that context, XPySOM is the more relevant external calibration baseline for the remaining results.

Observed QE differences are small in magnitude, typically below 1%, and are consistent with floating-point accumulation-order effects rather than algorithmic divergence. When the paired summaries visibly favor one implementation, the direction more often favors FloatSOM, but the effect size is generally not practically material for deployment-level decisions.

For runtime interpretation of this initial calibration, FloatSOM includes a small one-time startup cost from JIT kernel compilation. This overhead is most visible on small workloads, but is progressively amortized as sample count and workload size increase. On the largest benchmark datasets (covertypes and kddcup99), runtime is on par with or faster than XPySOM under this matched protocol, consistent with compilation-cost amortization. This trend is expected to strengthen further as dataset scale increases (Sections 6.1-6.2).

5.2 Comparison of Different Sampling Methods

We first report the focused HDSSSOM pilot as an elimination comparison rather than as part of the broader sampling benchmark. This pilot used a different run envelope from the later full-versus-random analysis: it was restricted to the hexagonal Optuna campaign with 10 datasets, 5 seeds, and `sampling_method \in \{full, random, hdsssom\}`; the corresponding pilot configuration is summarized in Supplementary Table S2.

Under this pilot configuration, HDSSSOM was not merely directionally worse but uniformly and materially worse than the other sampling options. In the hexagonal full-batch view shown in Fig. 2, full outperformed HDSSSOM in all 50 paired comparisons (10 datasets $\times$ 5 seeds; 50 wins, 0 losses, 0 ties), with dataset-level median balanced- QE improvements ranging from 2.5% to 209.7% and a GLOBAL_OVERALL median improvement of 38.7%. The matched random-versus-HDSSSOM comparison showed the same direction across all datasets and paired units, with a corresponding GLOBAL_OVERALL median improvement of 38.3%. This is what we mean here by “much worse”: the effect is consistent across datasets, unidirectional across all paired units, and large in magnitude rather than a marginal but statistically detectable shift. Because HDSSSOM failed at this initial screening step, we excluded it from the remaining experiments.

Supplementary Figure S4: Full vs HDSSSOM Outcomes (Hexagonal / Full Batch)

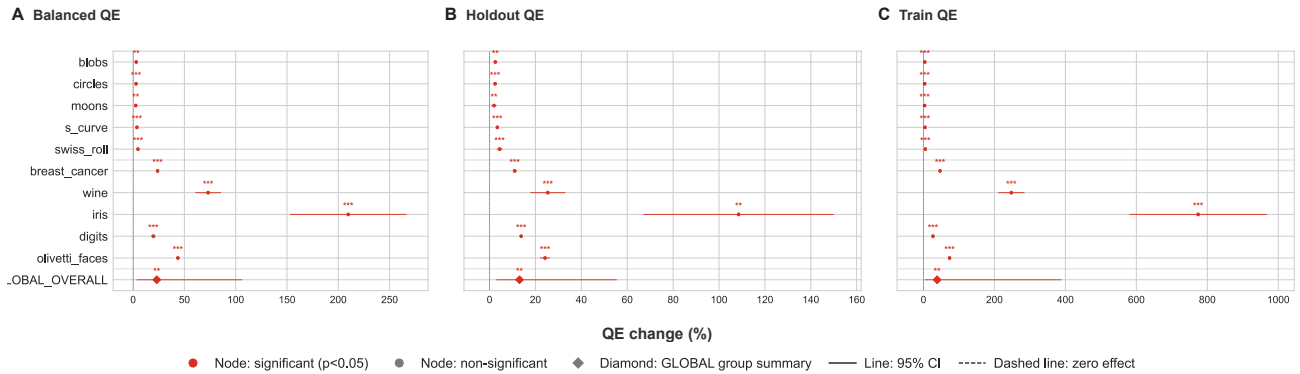


Figure 2. HDSSSOM screening pilot on QE_B (hexagonal topology): full vs HDSSSOM, using the restricted pilot configuration summarized in Supplementary Table S2 (10 datasets, 5 seeds, with all other pilot settings held fixed within that run envelope). Panels report dataset-matched paired top- k within-unit medians plus dataset-level paired-effect summaries (Section 4.3). Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect.

We therefore focus the remaining sampling question on full versus random, where the overall quality difference is much smaller and the practical tradeoff is between throughput and stability rather than between competitive and clearly non-competitive methods. We observe that the effectiveness of random sampling is scale-dependent: above 10,000 samples, paired QE differences are not

meaningfully detected, whereas in smaller datasets the random arm shows higher variability and less stable outcomes, consistent with reduced per-iteration sample support under random subsampling (Fig 3D-F), with the $> 10,000$ being a useful proxy for stable and unstable random schemes.

Figure 2: Full vs Random Outcomes and Dataset-Size Regression (Hexagonal / Full Batch)

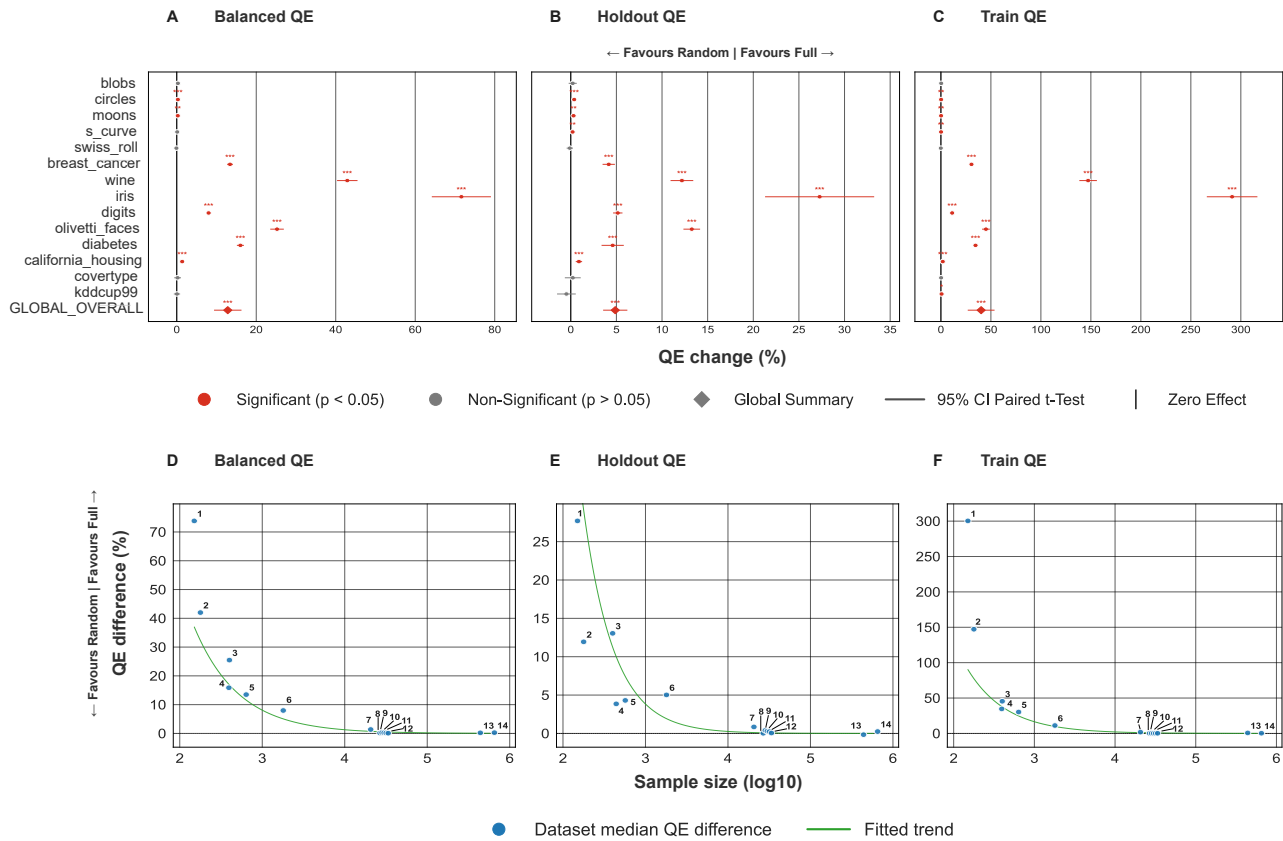


Figure 3. Sampling-mode comparison focused on full versus random under the paired analysis pipeline (Section 4.3). In the empirically larger-dataset regime observed here (> 10000 samples), little paired QE separation is detected; at smaller dataset scales, random is more variable and less stable. In the forest panels, whiskers denote 95% paired t-test confidence intervals around the mean paired effect. [[AUTO-SAMPLING-REGRESSION-STATS]]

However, as expected, full-sampling still remains the best sampling strategy for optimal QE results. Accordingly, all remaining analyses reported below rely on full-batch training unless specified.

5.3 Topology Results

Topology comparisons are reported with QE -only endpoints because distortion is unavailable for MST in this setup. We treat the Optuna hexagonal batch setting as the primary regular-topology baseline in this panel and compare MST and RNG against it.

To anchor the topology results qualitatively, Fig. 4 shows representative neighborhood overlays for hexagonal, MST, and RNG on a synthetic sklearn circles dataset. We use this panel only as geometric intuition for the topology comparisons that follow; the quantitative results remain the paired QE analyses in Figs. 5-6 and Fig. S4.

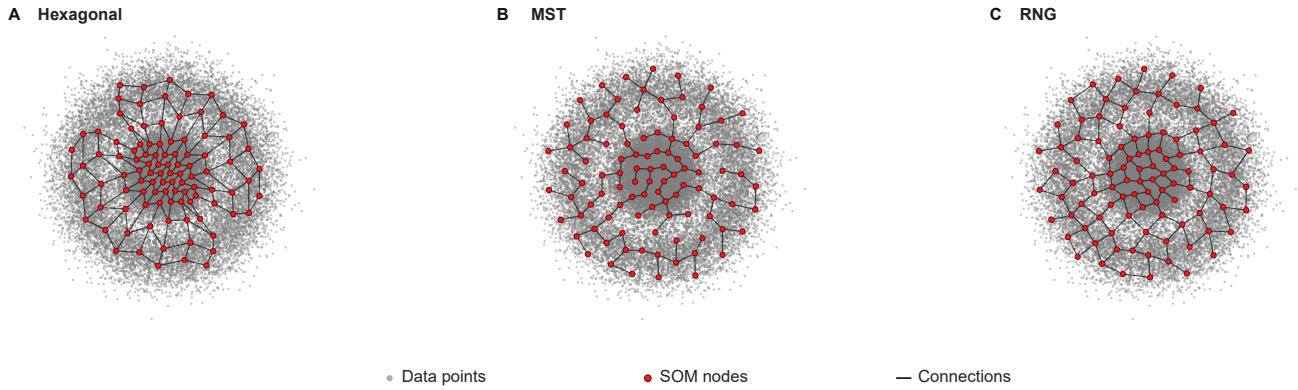
Figure 3: Representative Topology on Slearn Circles

Figure 4. Representative figure for XPySOM default run Hexagonal, MST, and RNG neighborhood node and connection overlays on a 30,000 datapoint synthetic sklearn circles dataset.

5.3.1 MST

We use dataset-wise paired improvement summaries (Hexagonal over MST) with the same reporting logic as Section 5.1, centered on QE_B . We report QE_H and QE_T separately for Hexagonal versus MST to expose train/holdout trade-offs. The main Hexagonal-versus-MST topology figure (Fig. 5) is an outcomes-only tripanel across QE_B , QE_H , and QE_T .

Overall, MST outperforms matched hexagonal on Balanced QE (Fig. 5A), indicating a net advantage across train and holdout performance. This aggregate gain is driven more clearly by Train QE (Fig. 5C), while Holdout QE is more mixed across datasets (Fig. 5B) and shows no clear overall holdout advantage. The overall paired t-test p-values are Balanced QE ($p=1.12e-05$); Holdout QE ($p=0.15$); and Train QE ($p=0.0064$). Supplementary Table S6 lists the per-dataset and overall hexagonal-comparison p-values for MST and RNG.

Across the tested top- k range, which varies the number of best-ranked retained trials per matched unit, the Hexagonal-versus-MST comparison remains directionally stable, indicating that the observed effect is not an artifact of a single pairing cutoff; the corresponding sensitivity analysis is provided in Fig. S5.

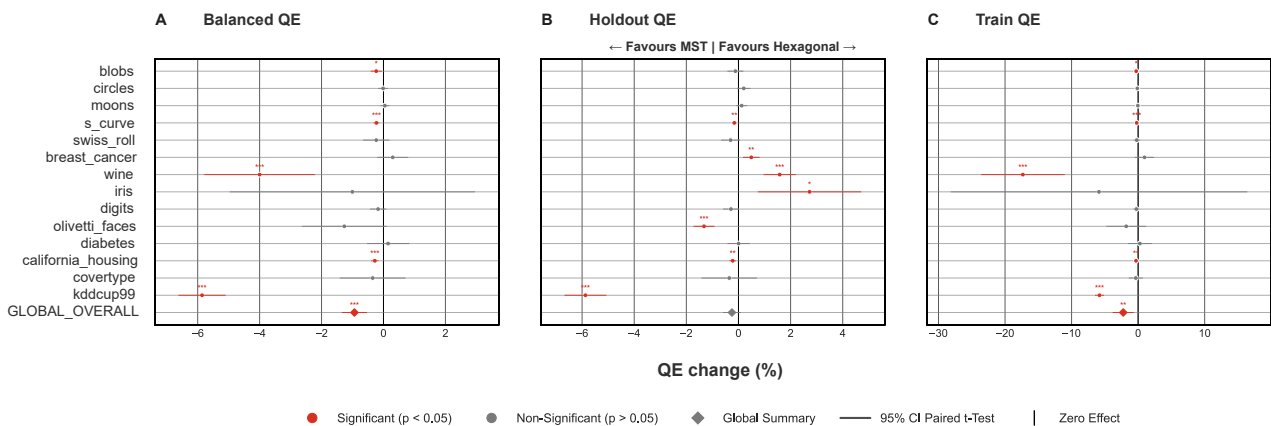
Figure 4: Hexagonal vs MST Across QE Metrics

Figure 5. Hexagonal versus MST topology on QE endpoints under full sampling only. Panels A-C report paired full-sampling-only QE effects for QE_B , QE_H , and QE_T across the available full-sampling datasets. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

For completeness, we also ran a default-setting FloatSOM MST configuration against XPySOM under the same seed-matched and train-holdout setup (Supplementary Fig. S1). This was intentionally done with XPySOM-equivalent default batch settings for equivalence calibration. Notwithstanding that these are inherited hexagonal hyperparameters, MST still outperforms the default hexagonal baseline, indicating *prima facie* that the MST topology is already favorable relative to the current regular-topology reference even before topology-specific tuning is applied.

5.3.2 RNG

To evaluate the RNG topology performance, we reuse the paired reporting logic on Hexagonal versus RNG, again centered on QE_B with QE_H and QE_T reported separately to expose train/holdout trade-offs. The main Hexagonal-versus-RNG topology figure (Fig. 6) is likewise an outcomes-only tripanel across QE_B , QE_H , and QE_T .

RNG has lower QE than matched hexagonal on the reported QE metrics (Fig. 6A-6C), with overall paired t-test p-values of Balanced QE ($p=7.4e-10$); Holdout QE ($p=0.0232$); and Train QE ($p=4.69e-06$). Supplementary Table S6 lists the per-dataset and overall hexagonal-comparison p-values for MST and RNG.

The main trend in Fig. 6 is that RNG improves on hexagonal most clearly in Balanced QE and especially in Train QE, rather than by producing an equally strong shift on every dataset. The separation tends to be more apparent in the real and larger datasets, where the added flexibility of the graph neighbourhood appears more useful than the fixed regular lattice. Holdout QE also shows a clear overall advantage for RNG, indicating that the improvement is not confined to training-set fit alone; rather, the same direction is retained on unseen data, even if the effect is smaller than in the training panel. Taken together, the figure suggests that RNG is generally better than hexagonal, with the clearest benefit appearing in training-set fidelity but with the holdout results supporting the same overall conclusion.

Similar to the MST results, across the tested top- k range, the Hexagonal-versus-RNG comparison likewise remains directionally stable; the corresponding sensitivity analysis is provided in Fig. S6. The direct MST-versus-RNG sensitivity comparison is reported separately in Fig. S7.

Figure 5: Hexagonal vs RNG Across QE Metrics

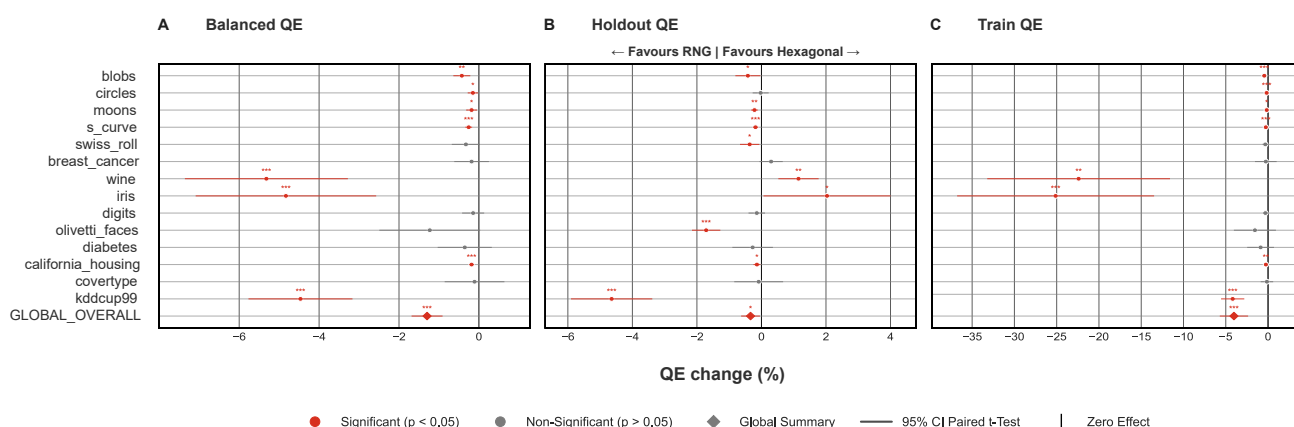


Figure 6. Hexagonal versus RNG topology on QE endpoints under full sampling only. Panels A-C report paired full-sampling-only QE effects for QE_B , QE_H , and QE_T across the available full-sampling datasets. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

These results are qualitatively consistent with Fig. 4, where RNG retains mesh-like local connectivity where geometry supports it while also allowing freer non-mesh edges in more irregular regions, unlike the fixed hexagonal lattice and the tree-constrained MST. Again, for completeness, we also ran a default-setting FloatSOM RNG configuration against XPySOM under the same seed-matched

and train-holdout setup (Supplementary Fig. S2). Even under those inherited hexagonal hyperparameters, RNG still outperforms the default hexagonal baseline, further indicating that the topology itself is favorable relative to the regular-topology reference even before topology-specific tuning is applied.

5.4 Hyperparameter Tuning and Stability

Hyperparameter effects enter the practical interpretation of these results in two ways. First, tuning can improve attainable QE relative to an untuned reference configuration. Second, the stability of the selected hyperparameters across seeds, topologies, and sampling modes determines how sensitive each method is to parameter choice in practice. We therefore separate the hyperparameter results into tuning benefit (Fig. 7) and hyperparameter stability (Fig. 8).

5.4.1 Performance Gains from Hyperparameter Tuning

The pooled tuned-versus-reference QE comparison is shown in Fig. 7, with topology-specific Hexagonal, MST, and RNG breakdowns provided in Fig. S8-10, respectively. Here, the tuned configuration is a fixed hyperparameter setting derived from the Optuna workflow by aggregating the selected tuned settings across seeds, using the mean for numeric parameters and the mode for categorical parameters, and then rerun on the datasets. This comparison therefore estimates the gain from adopting that tuned setting as the operating configuration, relative to an untuned reference configuration. We observe universal large QE improvements to hyperparameter tuned runs relative to un-tuned runs.

Figure 6: Tuned vs Default Across QE Metrics

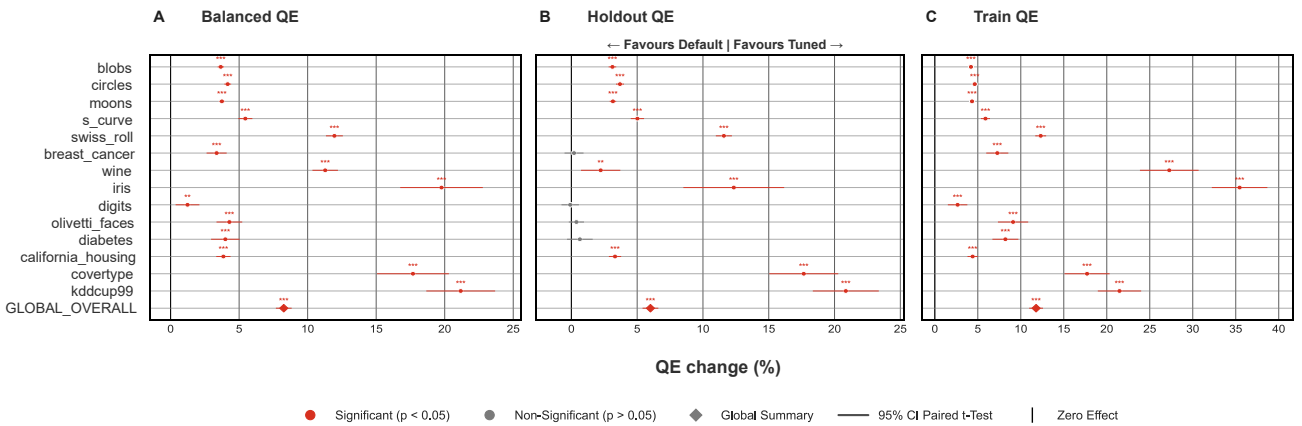


Figure 7. Tuned-configuration-versus-untuned-reference QE comparison across QE_B , QE_H , and QE_T , pooled across all topology runs under the matched pairing keys. Positive values indicate the tuned configuration outperforms the untuned reference; the global overall row pools all matched tuned-configuration/untuned-reference pairs across datasets. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

In Fig. 7, we report both the untuned-minus-tuned paired difference, so positive values favor the tuned configuration for lower-is-better metrics, and the corresponding percent improvement relative to the untuned reference value. The tuned-versus-reference pairing results in Fig. 7 show the same direction across the QE endpoints, based on $n=840$ paired comparisons per QE endpoint ($n=2,520$ total across all QE variants $QE_B/QE_H/QE_T$) from 14 datasets, 10 seeds, and the full and random sampling modes. Across the matched pairs, tuned settings improve Balanced QE in 786/840 pairs (54 worse), with median and mean improvements of 5.00% and 8.26%; Holdout QE in 675/840 pairs (165 worse), with median and mean improvements of 3.47% and 6.01%; and Train QE in 799/840 pairs (41 worse), with median and mean improvements of 7.58% and 11.78%.

At the pooled overall level, the paired summaries across all matched tuned-configuration/untuned-reference pairs also favor tuning for all three metrics, consistent with the per-dataset pattern in Fig. 7.

The same tuning pattern is observed across topologies: mean Balanced-QE improvement is positive for hexagonal (10.35%), MST (6.46%), and RNG (7.97%), indicating that tuning affects all topology families rather than a single-architecture artifact.

5.4.2 Hyperparameter Stability Across Topology and Sampling

Hyperparameter stability analyses (Section 4.5) compare within-topology seed-to-seed tuned-parameter drift and then contrast those internal-stability scores between topology pairs. The resulting pattern indicates that MST and RNG reach lower stability scores than hexagonal when matching dataset, sampling mode, and seed structure. Fig. 8A summarizes the full-sampling stratum, and Fig. 8B shows the corresponding random-sampling analysis. The full-versus-random contrast is also directional: the full-sampling panel generally shows lower selected-parameter stability scores than the random-sampling panel for the same topology families, suggesting modestly better hyperparameter stability under full sampling. The dataset-size regression summaries show little evidence of a full-sampling size relationship, with near-zero correlations under full sampling, hexagonal (Pearson $R=0.082$, $p=0.781$, $n=14$); MST (Pearson $R=0.198$, $p=0.497$, $n=14$); and RNG (Pearson $R=-0.192$, $p=0.511$, $n=14$). By contrast, Fig. 8C shows a clearer random-sampling size relationship, with hexagonal (Pearson $R=-0.820$, $p=0.000326$, $n=14$); MST (Pearson $R=-0.839$, $p=0.000176$, $n=14$); and RNG (Pearson $R=-0.512$, $p=0.0613$, $n=14$). Under random sampling, larger datasets tend to produce lower selected-parameter stability scores, indicating improved stability with scale. This reinforces the practical interpretation that random is attractive primarily as a throughput-oriented choice rather than a stability-first setting at smaller dataset scales.

Figure 7: Hyperparameter Stability

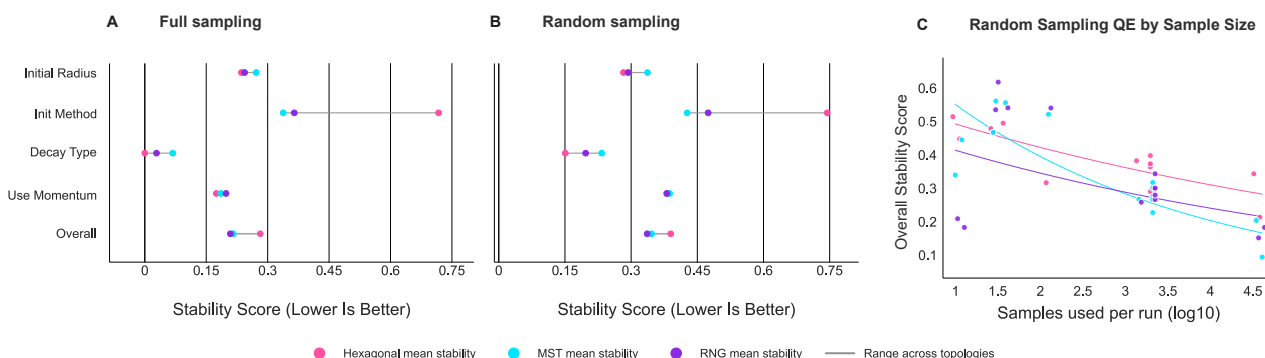


Figure 8. Hyperparameter stability by sampling mode. A: selected-parameter stability under full sampling for hexagonal, MST, and RNG topologies (lower stability score is better). B: selected-parameter stability under random sampling for the same topologies. C: dataset-size stability regression under random sampling, using the selected-parameter stability score against sample size ($\log_{10}$) across the included topology families.

Overall, the stability results indicate that full sampling remains the more stable choice, while random sampling becomes more viable as dataset size increases. Within that picture, MST and RNG show lower selected-parameter stability scores than hexagonal across the matched comparisons, suggesting that the graph topologies are less sensitive to tuning variation even when subsampling is introduced.

6. Speed Scaling

This section addresses three practical questions for deployment: whether choosing `mst` or `rng` imposes a meaningful scalability penalty relative to `hexagonal`, how much runtime is added by `full` relative to `random` sampling, and what additional GPUs buy in practice. Across these results, topology choice is not the dominant runtime cost in the main sample- and dimension-scaling

regimes; the clearer speed tradeoff is sampling, where `random` is faster than `full`. Additional GPUs matter most because they expand feasible workload size and reduce memory-pressure limits, which is especially relevant for `full` runs.

6.1 Random versus full sampling runtime

We first isolate the most direct runtime tradeoff, namely the cost of using `full` rather than `random` sampling. In the runtime comparison (Fig. 9), sample-scaling runtime is shown for full versus random sampling in batch mode, with topology traces for hexagonal, MST, and RNG.

Figure 8: Sampling Runtime

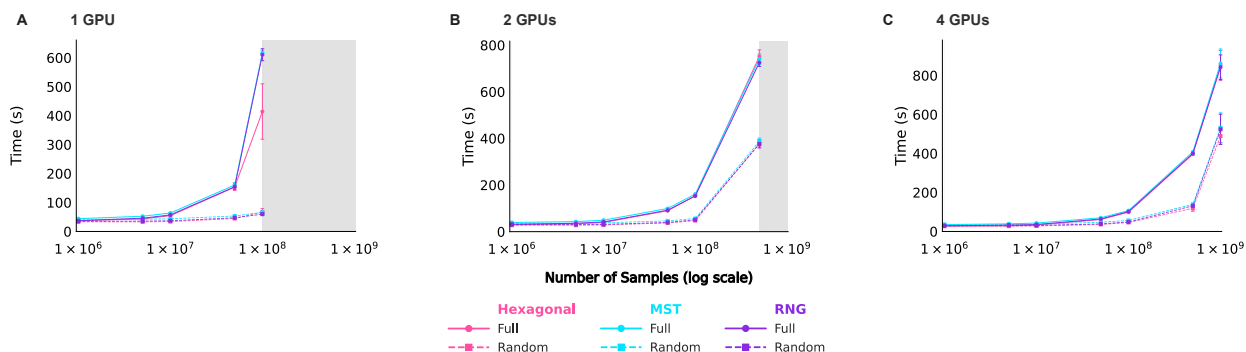


Figure 9

Figure 9. Sample-scaling runtime comparison of full versus random sampling in batch mode. Curves report mean wall-clock training time (s) under harmonized settings; error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration. Color encodes topology (hexagonal, MST, RNG), and line style encodes sampling mode (full vs random). Shaded x-axis regions indicate sample-size ranges not able to be run in that panel relative to the shared axis maximum, due to timeouts. Lower values indicate faster execution.

Across topologies, random sampling trends faster than full sampling in this harmonized 1,2,4-GPU comparison, with similarly proportioned reductions seen across all GPU configurations at any given dataset size. Larger GPU counts also extend the largest dataset size that can be processed before the 30-minute timeout is reached, so the benefit of adding GPUs is not only shorter runtime but a wider feasible workload range. Consequently, combined with the *QE* evidence in Fig. 2, this supports a scale-aware operational interpretation: above 10,000 samples, random can be used without tangible paired *QE* loss relative to full, whereas for smaller datasets full remains the more stable choice.

One important exception is the single-GPU large-sample regime. In practice, the last successful 1-GPU random point occurs at 100M samples, and the next point at 500M fails under the benchmark timeout, despite random otherwise being the faster sampling mode. We interpret that failed 500M point as the stage at which the single-GPU path has tipped into disk-backed operation, so the relevant cost is no longer only the reduced number of selected samples but the overhead of staging data to disk and transferring them through the single-GPU path. We therefore treat the 1-GPU random failure at 500M samples as a disk-mode systems limitation rather than as evidence against the general random-versus-full runtime ordering. A plausible explanation, which we treat as a systems-level interpretation rather than a directly isolated measurement, is that the single-GPU path does not provide enough effective bandwidth for disk-mode operation to remain efficient in the same way as the 2+ GPU configurations.

This establishes the main speed penalty associated with choosing the more faithful data regime: for a fixed topology, `full` takes longer than `random` at the same sample size. The next question is therefore not whether random is faster, but how much of the practical burden of `full` can be offset by additional GPUs.

6.2 Multi-GPU topology scaling and OOM context

We next examine what additional GPUs buy at the systems level. To maximally stress-test the implementation, we employ harmonized full-batch processing across $G \in \{1, 2, 4, 8\}$ GPUs and evaluate performance in the regime where RAM is no longer the limiting factor, allowing us to probe the computational limits leading up to out-of-memory (OOM) conditions. Under these conditions, runtime and efficiency exhibit consistent scaling behavior across workloads (Fig. 10). The practical timeout boundary also shifts with GPU count in the sample-scaling panel: the 1-GPU configuration times out at 500M samples, whereas the 2-GPU configuration reaches 500M but times out at 1B under the benchmark wall-clock limit.

Figure 9: Multi-GPU Full-Batch Scaling (RNG)

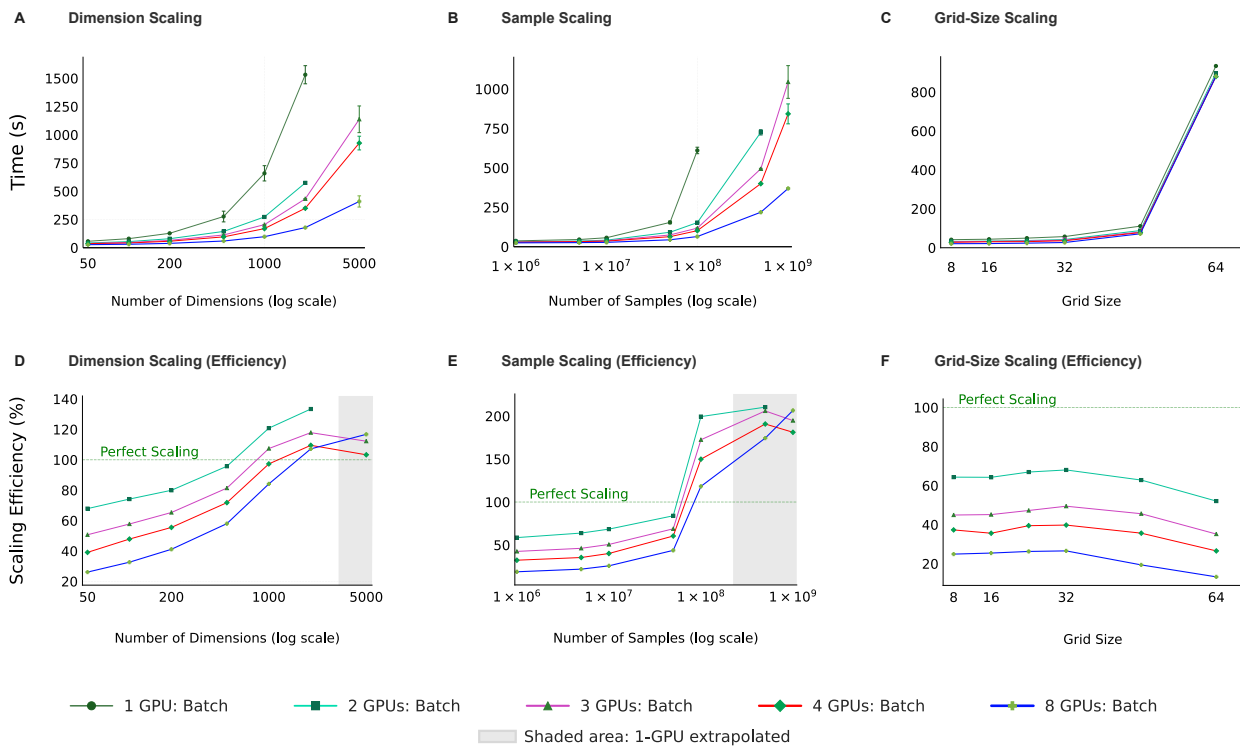


Figure 10

Figure 10. Multi-GPU full-batch scaling across $G \in \{1, 2, 4, 8\}$ GPUs. Panels A-C show runtime (s) for dimension-, sample-, and grid-size-scaling workloads, respectively. Panels D-F show scaling efficiency for the same workloads, computed from the single-GPU baseline and the corresponding G -GPU runtime. Runtime error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration; the 100% efficiency reference line indicates ideal linear scaling.

The available staging diagnostics show that increasing GPU count changes the sample-scaling regime in two ways. First, disk-backed execution is deferred to larger workloads as the aggregate worker-memory pool grows: in sample scaling, the RNG path is already disk-backed at 2 GPUs for 500,000,000 samples, whereas the 8-GPU configuration remains in RAM at that workload and does not fully enter disk mode until 1,000,000,000 samples. Second, when disk-backed staging is

required, higher GPU counts still improve runtime because staging and disk-to-GPU transfers are distributed across more workers rather than saturating a narrower transfer path. The 8-GPU RNG configuration processes 1,000,000,000 samples in 369.41 s (6.16 min), demonstrating billion-sample training at a runtime measured in minutes rather than hours even for a 50-feature, 1024-node network ($32 \times 32 = 1024$) under multi-node distributed execution with data staged from shared non-local storage to node-local shards before training. This links the Fig. 9 random-versus-full result to the multi-GPU/OOM scaling result on the same 1,000,000,000-sample RNG workload: at 4 GPUs, moving from random to full adds 369.48 s (6.16 min) (+63.41%), but the 8-GPU full configuration finishes that same workload 61.20% faster than the 4-GPU full run and 36.60% faster than the 4-GPU random run. The practical gain from scaling out is therefore not only more compute, but also a larger effective memory pool and a wider staging-bandwidth path, both of which reduce the runtime burden of full-sampling OOM execution. The grid-size panel is the main exception: at the largest tested grid size (64), runtime changes only from 934.01 s (15.57 min) on 1 GPU to 880.83 s (14.68 min) on 8 GPUs, a 5.69% reduction, indicating that once map-size/topology-refresh costs dominate, additional GPUs contribute little extra speedup.

We next turn from absolute runtime to GPU efficiency under strong scaling, asking how closely additional GPUs approach ideal linear speedup on the same workload. We interpret scaling efficiency relative to ideal linear scaling, such that 100% efficiency means that doubling or octupling the GPU count yields the corresponding 2x or 8x runtime reduction, while lower values indicate increasing overhead from communication, orchestration, or I/O. In Fig. 10D-F, efficiencies are high at the higher GPU counts, including values above 100%. When a direct 1-GPU baseline was unavailable at a given axis value, the efficiency denominator was constructed by local linear extrapolation from the last available 1-GPU point on that curve (Section 4.2), so the exact magnitude of some values should be interpreted with care if the underlying 1-GPU runtime is nonlinear over that range. Fig. 10 shows the scaling for the RNG topology, while Fig. S11 shows corresponding supplementary Hexagonal and MST outputs, which show the same super-linear pattern.

This scaling context clarifies the practical meaning of Fig. 9. The random-versus-full runtime penalty is real at fixed GPU count, but it is not the whole systems story: scaling out can absorb much of that penalty by postponing disk spill into a larger aggregate RAM pool and, when disk mode is still required, by distributing the staging path across more workers. The main exception is grid-size scaling, where larger maps show little benefit from additional GPUs because topology-refresh and map-size costs dominate. In practice, the biggest gain from additional GPUs is therefore not only faster execution, but the ability to keep more demanding full configurations feasible at larger sample sizes without letting wall-clock time drift back into the multi-hour regime.

With that systems context in place, we next compare topology runtimes across hexagonal, MST, and RNG configurations on 8 GPUs (Fig. 11).

Figure 10: Topology Runtime Comparison (8 GPUs, Full Batch)

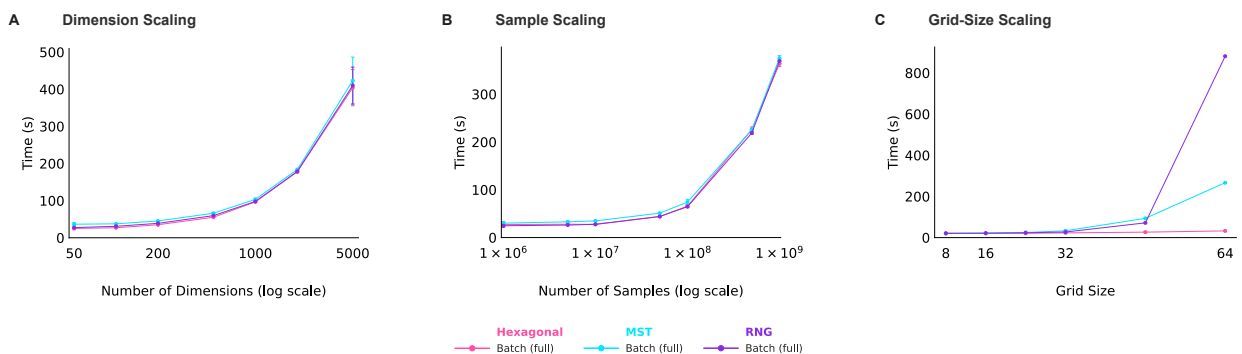


Figure 11

Figure 11. Topology runtime comparison at fixed $G = 8$ GPUs under full-batch processing. Panels A-C report mean wall-clock runtime (s) for dimension-, sample-, and grid-size-scaling workloads, respectively, with topology traces for hexagonal, MST, and RNG. Error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration. The largest-axis 8-GPU topology runtime summaries are listed in Supplementary Table S10.

In Fig. 11A-B, the topologies scale similarly as input complexity and data volume increase: even at the largest tested axis values, the maximum pairwise runtime spread remains modest at dimension scaling (4.70% at 5,000 dimensions); sample scaling (3.26% at 1,000,000,000 samples).

However, when the grid itself is enlarged in Fig. 11C, topology-dependent runtime differences become readily evident. At the largest tested grid size (grid size 64), the 8-GPU mean runtimes are 32.54 s (0.54 min) for hexagonal, 266.45 s (4.44 min) for MST, and 880.83 s (14.68 min) for RNG, corresponding to 8-GPU MST and RNG runtimes that are 8.19x and 27.07x the hexagonal runtime, respectively.

Taken together, these topology comparisons show that `mst` and `rng` remain broadly runtime-competitive with `hexagonal` in the main sample- and dimension-scaling regimes, so topology choice is not the dominant speed bottleneck there. The clearer caveat is that topology-dependent costs become more visible when the map itself is enlarged, especially for RNG. The practical implication is that the main runtime tradeoff for most large-data use remains `random` versus `full`, while topology becomes the larger systems concern in grid-size-dominated settings.

Taken together, Figs. 10-11 show that FloatSOM's systems contribution is twofold: efficient distributed scaling when additional GPUs and nodes are available, and a genuinely usable OOM-capable execution mode when they are not. In that sense, the OOM path is not merely a safeguard against memory failure, but a practical alternative to full-RAM operation for very large datasets under constrained GPU availability.

7. Discussion

With the scaling story established, Fig. 12 then tests whether the *QE* gains from topology choice and tuning persist in deployment against XPySOM, a current high-performance Python SOM baseline in this benchmark context (Mancini et al. 2020).

Figure 11: XPySOM (Default) v FloatSOM RNG

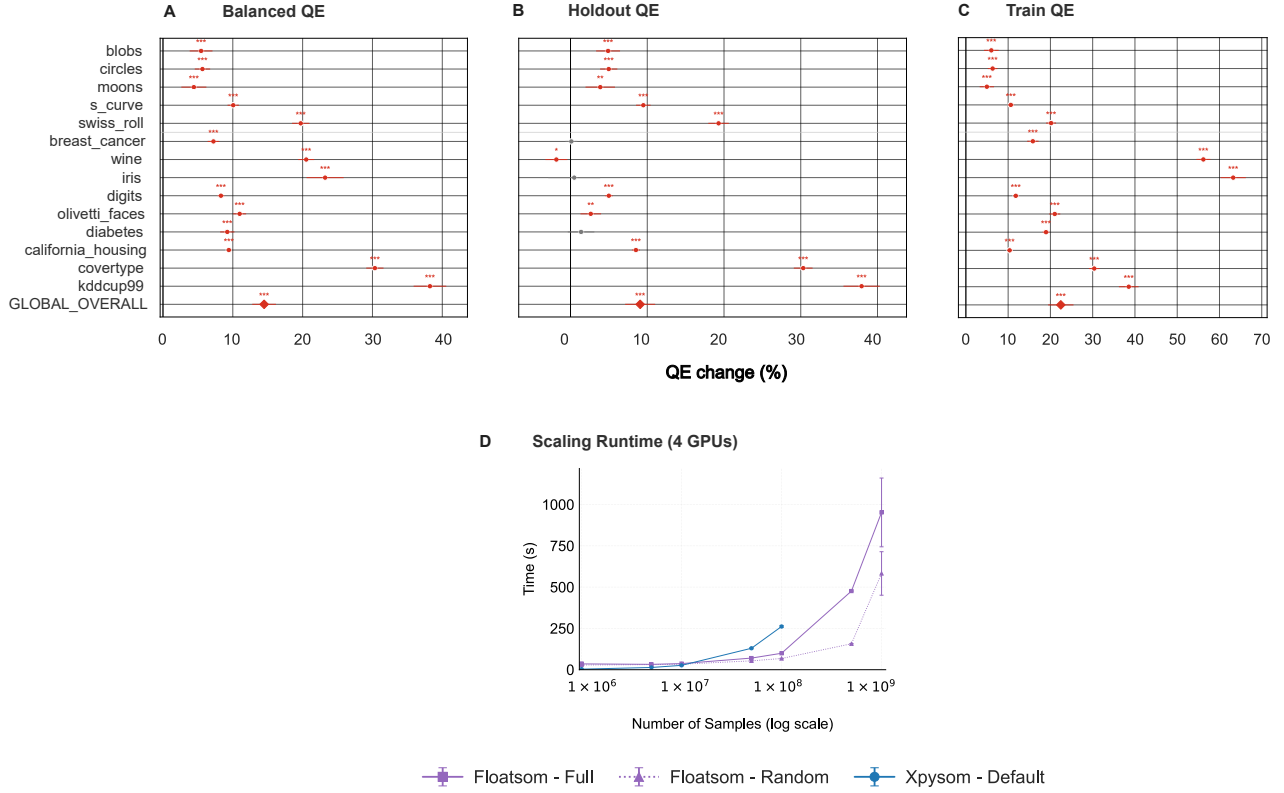


Figure 12

Figure 12. Integrated deployment comparison of default hexagonal XPySOM versus tuned FloatSOM RNG. Panels A-C compare QE_B , QE_H , and QE_T using the untuned hexagonal XPySOM baseline against matched tuned FloatSOM RNG full-sampling runs. Panel D provides the scaling/runtime context for the same comparison, with the separately executed targeted 1B-sample runs discussed in the text rather than plotted directly. Taken together, this integrated figure summarizes the operating point observed for tuned FloatSOM RNG once workload size is large enough for steady-state execution to dominate startup overhead. Per-dataset and GLOBAL_OVERALL panel summaries are listed in Supplementary Table S7.

At the overall level, Fig. 12 shows median percentage improvements of QE_B (14.5%); QE_H (9.1%); and QE_T (22.5%) for tuned FloatSOM RNG relative to default hexagonal XPySOM, capturing the combined deployment effect of topology choice and tuning on QE .

For the default hexagonal XPySOM reference in Fig. 12, workloads beyond the 10^8 -sample case were not processed under this benchmark setup because the implementation ran out of GPU VRAM and required the full dataset to be loaded into memory. This deployment comparison therefore provides a concrete endpoint for the speed-scaling story above: the main systems difference is not merely raw runtime at matched scale, but the ability to sustain larger workloads under distributed, out-of-memory-capable execution.

In the deployment comparison, tuned FloatSOM RNG delivers better QE than the default hexagonal XPySOM baseline, while also running faster and scaling to larger workloads (Supplementary Table S10).

7.1 Sampling tradeoff (random versus full)

These results suggest that the sampling trade-off is strongly scale dependent. In smaller datasets, random subsampling appears to increase update variance because each iteration is supported by fewer observations, which makes outcomes less stable. As dataset size grows, that instability seems to weaken, and under iteration-matched training the quality gap between full and random sampling largely disappears. From this perspective, full sampling is the safer choice when stability is the priority in smaller datasets, whereas random sampling becomes the more practical option when throughput is the dominant concern at larger scales.

This interpretation should be qualified in the very-large-dataset regime ($>$ RAM capacity). There, the dominant bottleneck shifts away from pure compute and toward disk-read behavior and transfer into node-local storage. In the current implementation, the random path still reads each worker-local chunk before subsampling within it. Distributed shards still need to be cloned in their entirety to their respective workers, therefore, incurring the same dataset IO costs as ‘full’ sampling. A logical direction for future work would be a chunk-level randomization path that avoids the current full-chunk-read requirement, but that approach is outside the validated scope of the present study and may introduce different update behavior. Even so, the present random path remains faster in this regime and is still roughly an order of magnitude faster than the corresponding full-data path at the largest scales considered.

The single-GPU random crossover between the last successful 100M point and the failed 500M point helps clarify that caveat. At that failed 500M point, the dominant cost appears to have shifted to file transfer and staging overhead rather than the nominal sampling fraction itself. Our interpretation is that single-GPU bandwidth is insufficient for disk-backed operation to behave as efficiently as it does under 2+ GPU execution, where the staging and transfer burden is distributed more effectively. We treat this as a plausible explanation of the observed runtime pattern rather than as an independently benchmarked bandwidth result.

7.2 Topology tradeoff (MST and RNG)

The MST result appears to be driven primarily by training-set fidelity rather than by a uniform holdout advantage. One possible interpretation is that the tree-structured neighborhood conforms more efficiently to the occupied training manifold, allowing MST to represent the observed sample geometry with lower Train QE than the fixed hexagonal lattice. The same efficiency may also make MST less forgiving when holdout samples occupy somewhat different regions of feature space, which could explain the mixed Holdout-QE pattern across datasets. In those cases, the more uniform spatial coverage imposed by the hexagonal lattice may provide a modest generalization buffer. We treat this as an interpretation rather than a demonstrated mechanism, because the present benchmark does not explicitly quantify train-to-holdout distribution shift.

The topology results point to a simple trade-off. Hexagonal neighborhoods are the most restrictive because they impose a fixed mesh (Kohonen 2013); MST relaxes that structure but still limits propagation to tree paths (Kangas et al. 1990); RNG allows denser graph connectivity as prototype geometry evolves (Toussaint 1980). This pattern is consistent with the lower QE observed for RNG in our results, although the present comparisons do not isolate the connectivity mechanism directly. One interpretation of this difference is RNG’s ability to support both topology regimes within one model: mesh-like local organization where manifold structure is regular, and freer non-mesh connectivity where the manifold is irregular or partially disconnected. In contrast, hexagonal neighborhoods enforce a fixed mesh and MST enforces a strictly tree-constrained structure. Because the RNG graph is recomputed over moving prototypes during topology refresh, the neighborhood structure can shift between these two regimes over training rather than remaining fixed. The same qualitative ordering is robust across the tested top- k sensitivity range, supporting the interpretation that the observed topology effect is stable rather than an artifact of a single pairing cutoff; the

corresponding topology-comparison sensitivity analyses are provided in Supplementary Figs. S5-S7. The main downside of RNG is its less favorable topology-refresh scaling at very large node counts, so its gap narrows in grid-size-dominated regimes. The expensive step is the relative-neighborhood edge test itself: for each candidate prototype pair (i, j) , RNG checks every third prototype k to determine whether k blocks that edge by being closer to both endpoints than the endpoints are to each other, that is, whether $\max(d(i, k), d(j, k)) < d(i, j)$. In other words, the implementation must examine prototype triplets (i, j, k) rather than prototype pairs alone. For P prototype nodes, that edge-construction stage therefore carries a worst-case cost of $O(P^3)$ (Toussaint 1980). Because these benchmarks use square maps, increasing grid size from g to a larger side length increases the prototype count to $P = g^2$, so the number of potential triplet checks rises very rapidly as the map grows. In the implemented FloatSOM path, however, both MST and RNG then pass through the same all-pairs shortest-path precomputation during topology refresh, so the end-to-end refresh cost is not separated cleanly by the edge-construction terms alone. The safer reading of these results is therefore empirical rather than purely asymptotic: RNG gives the strongest QE in these comparisons, while MST remains a reasonable alternative when very large maps make the RNG topology-refresh stage the practical bottleneck.

7.3 Tuning benefit under matched defaults

Across the matched Fig. 7 comparisons, tuned configurations consistently produce better QE results than untuned reference settings. This suggests that tuning should be treated as part of the method configuration rather than as optional post-processing.

The key interpretation is that topology choice and hyperparameter choice are coupled. Gains remain positive across hexagonal, MST, and RNG, so tuning is not confined to a single topology. This is also consistent with the broader hyperparameter-optimization literature, where achieved performance depends materially on the search process and the selected configuration rather than on architecture alone (Bergstra et al. 2011). A transferable deployment strategy therefore requires topology-aware tuning rather than a single universal setting.

7.4 Hyperparameter stability and dataset-type interpretation

The stability analysis adds an operational layer to the quality results. Hexagonal maps appear more constrained because their neighborhood structure is fixed by the initial lattice: if those initial connections are poorly aligned with the data geometry, training can move the prototypes but cannot rebuild the connectivity itself. That makes the final outcome more sensitive to the initialization methodology and to seed-level variation in the tuned region. By contrast, MST and especially RNG recompute connectivity from the evolving prototype configuration, so they can begin from a broadly suitable initialization and then adapt the neighborhood structure as training proceeds. This likely makes the graph topologies easier to recover consistently in the tuned region and helps explain why they may be more suitable for higher-dimensional, non-synthetic datasets, where imposing a fixed low-dimensional lattice prior is more likely to create a geometric mismatch (Kohonen 2013; Kangas et al. 1990).

Cumulatively, we demonstrate robustly that MST and, especially, RNG better capture data topology than the fixed hexagonal baseline. They present a more stable set of hyperparameters, able to operate over multiple datasets and configurations.

7.5 Systems implications and limits

Distributed execution should therefore be interpreted as a workload-dependent regime change rather than as a uniform multiplicative speedup. The Ray-enabled path introduces fixed startup and orchestration costs, which can offset its benefits at small problem sizes. At larger workloads, however, the high efficiencies observed in Fig. 10 and the topology-specific scaling outputs are more

plausibly explained by changes in memory residency and data movement than by compute scaling alone. As GPU count increases, the dataset is partitioned into smaller worker-local shards, which reduces per-worker memory pressure and allows some workloads to remain in RAM that would otherwise spill to disk. When disk-backed staging is still required, assigning shards to workers localizes prefetch and transfer activity, so disk-to-GPU traffic is distributed across more workers rather than repeatedly contending for a narrower path. Communication overhead is likewise amortized because global collectives occur once per iteration, after local chunk accumulation, rather than after each chunk. The exact efficiency magnitudes should nevertheless be interpreted cautiously, because some 1-GPU baselines were obtained by local linear extrapolation and may be inaccurate where the single-GPU curve is nonlinear.

The random-sampling scaling behavior should be interpreted separately from that super-efficiency pattern. In the current implementation, the random sampling method still reads each touched worker-local chunk before subsampling within that chunk. Any data-cloning cost is limited to the worker-assigned shard, so the scaling behavior of the random sampling method remains tied to the same shard-read, I/O, and staging path even though fewer samples are ultimately used, which is why it does not preserve the stronger near-linear speedup seen at smaller scales. A natural future optimization would be a chunk-level randomization scheme, but that should be treated as a distinct method rather than as the validated random sampling method used here because it would avoid the current full-chunk-read requirement by restricting randomness to loaded chunks and may therefore produce different update behavior. Even so, the presently implemented random sampling method still yields runtime improvements that remain on the order of magnitude at the largest scales considered.

The practical implication is that multi-GPU execution becomes most useful once workload size is large enough for memory pressure and steady-state throughput to dominate orchestration overhead. In small workloads, distributed overhead can outweigh those benefits; in large workloads, scaling out is usually preferable because it sustains the end-to-end data path more effectively, even when both settings are disk-backed. This same interpretation explains the comparison with XPySOM: startup costs matter at small scale, but once those costs are amortized the distributed/OOM execution path becomes more relevant. Relative to current high-performance single-process baselines, the main systems difference is therefore one of scale as well as speed: under multi-GPU HPC execution, distributed aggregation and disk-backed streaming extend the feasible training regime beyond fixed in-memory limits, so practical workload size is constrained primarily by storage and I/O rather than by single-device memory. This pattern does not extend uniformly to all workloads, however: in the largest grid-size regime, runtime changes only from 934.01 s on 1 GPU to 880.83 s on 8 GPUs at grid size 64, showing that once map-size and topology-refresh costs dominate, additional GPUs provide little extra benefit. This manuscript reports four main findings: in iteration-matched comparisons, full and random show no meaningful paired QE difference in larger datasets (>10000 samples), while random provides runtime gains on smaller datasets, where it also shows greater random instability; graph topologies show lower QE than fixed hexagonal structure, with RNG showing the lowest QE in these comparisons; default-aware analyses over $n=840$ paired comparisons per QE endpoint ($n=2,520$ total across all QE variants $QE_B/QE_H/QE_T$), drawn from 14 datasets, 10 seeds, and the full and random sampling modes, show that hyperparameter selection affects outcomes under the derived default hyperparameters; and the multi-GPU, OOM-capable execution pipeline scales effectively when storage and file I/O are sufficient to sustain throughput.

Taken together, one operating profile supported by these experiments uses the maximum GPU count permitted by file I/O, RNG topology, and the derived default hyperparameters, with sampling chosen by scale: full for smaller datasets when stability is critical, and random as a throughput-oriented option in the empirically larger-dataset regime observed here (>10000 samples) where paired QE differences are not meaningfully detected. When workloads are dominated by very large grid-size scaling, MST remains a reasonable alternative because its graph-construction path scales more favorably than RNG.

More broadly, FloatSOM provides a GPU-oriented SOM implementation with a broad configuration space across sampling strategy, topology, batch/processing mode, and systems-scale execution. This design allows practitioners to select configurations aligned with different quality-throughput tradeoffs under deployment constraints. The study brings these components into a single evaluation framework.

8. References

- Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. *Optuna: A Next-generation Hyperparameter Optimization Framework*. arXiv:1907.10902. arXiv. <https://doi.org/10.48550/arXiv.1907.10902>.
- Akinduko, Ayodeji A., Evgeny M. Mirkes, and Alexander N. Gorban. 2016. “SOM: Stochastic Initialization Versus Principal Components.” *Information Sciences* 364–365 (October): 213–21. <https://doi.org/10.1016/j.ins.2015.10.013>.
- Alahakoon, Daminda, Saman Halgamuge, and Srinivasan Bala. 2000. “Dynamic Self-Organizing Maps with Controlled Growth for Knowledge Discovery. *IEEE Transactions on Neural Networks*, 11(3), 601–614.” *Neural Networks, IEEE Transactions on* 11 (June): 601–14. <https://doi.org/10.1109/72.846732>.
- Aoki, Takaaki, and Toshio Aoyagi. 2007. “Self-Organizing Maps with Asymmetric Neighborhood Function.” *Neural Computation* 19 (9): 2515–35. <https://doi.org/10.1162/neco.2007.19.9.2515>.
- Bergstra, James, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. “Algorithms for Hyperparameter Optimization.” *Advances in Neural Information Processing Systems* 24.
- Forest, Florent, Mustapha Lebbah, Hanane Azzag, and Jérôme Lacaille. 2020. “A Survey and Implementation of Performance Metrics for Self-Organized Maps.” In *arXiv.org*. <https://arxiv.org/abs/2011.05847v1>.
- HPC Systems NCI. n.d. <https://nci.org.au/our-systems/hpc-systems>.
- Ito, Keiichi, Ivo Couckuyt, Roberto d’Ippolito, and Tom Dhaene. 2016. “Design Space Exploration Using Self-Organizing Map Based Adaptive Sampling.” *Applied Soft Computing* 43 (June): 337–46. <https://doi.org/10.1016/j.asoc.2016.02.036>.
- Jang, Yoo-Jin, Myung-Hoe Huh, and Mi-Ra Park. 2009. “Use of Minimal Spanning Trees on Self-Organizing Maps.” *Korean Journal of Applied Statistics* 22 (January). <https://doi.org/10.5351/KJAS.2009.22.2.415>.
- Kangas, J. A., T. K. Kohonen, and J. T. Laaksonen. 1990. “Variants of Self-Organizing Maps.” *IEEE Transactions on Neural Networks* 1 (1): 93–99. <https://doi.org/10.1109/72.80208>.
- Kohonen, T. 1990. “The Self-Organizing Map.” *Proceedings of the IEEE* 78 (9): 1464–80. <https://doi.org/10.1109/5.58325>.
- Kohonen, Teuvo. 2001. *Self-Organizing Maps*. Edited by Thomas S. Huang, Teuvo Kohonen, and Manfred R. Schroeder. Vol. 30. Springer Series in Information Sciences. Springer. <https://doi.org/10.1007/978-3-642-56927-2>.
- Kohonen, Teuvo. 2013. “Essentials of the Self-Organizing Map.” *Neural Networks: The Official Journal of the International Neural Network Society* 37 (January): 52–65. <https://doi.org/10.1016/j.neunet.2012.09.018>.
- Kratochvíl, Miroslav, Oliver Hunewald, Laurent Heirendt, et al. 2020. “GigaSOM.jl: High-performance Clustering and Visualization of Huge Cytometry Datasets.” *GigaScience* 9 (11): g127. <https://doi.org/10.1093/gigascience/giaa127>.
- Kruskal, Joseph B. 1956. “On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem.” *Proceedings of the American Mathematical Society* 7 (1): 48–50. <https://doi.org/10.1090/S0002-9939-1956-0078686-7>.
- Liu, Candace C., Noah F. Greenwald, Alex Kong, et al. 2023. “Robust Phenotyping of Highly Multiplexed Tissue Imaging Data Using Pixel-Level Clustering.” *Nature Communications* 14 (1): 4618. <https://doi.org/10.1038/s41467-023-40068-5>.
- Mancini, Riccardo, Antonio Ritacco, Giacomo Lanciano, and Tommaso Cucinotta. 2020. “XPySom: High-Performance Self-Organizing Maps.” *2020 IEEE 32nd International*

- Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, September, 209–16. <https://doi.org/10.1109/SBAC-PAD49847.2020.00037>.
- Moritz, Philipp, Robert Nishihara, Stephanie Wang, et al. 2018. “Ray: A Distributed Framework for Emerging AI Applications.” *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (USA)*, OSDI’18, October, 561–77.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. “Scikit-Learn: Machine Learning in Python.” *Journal of Machine Learning Research* 12 (85): 2825–30.
- Spanakis, Gerasimos, and Gerhard Weiss. 2016. “AMSOM: Adaptive Moving Self-organizing Map for Clustering and Visualization.” In *arXiv.org*. <https://arxiv.org/abs/1605.06047v1>. <https://doi.org/10.5220/0005704801290140>.
- Toussaint, Godfried T. 1980. “The Relative Neighbourhood Graph of a Finite Planar Set.” *Pattern Recognition* 12 (4): 261–68. [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7).
- Vasighi, Mahdi, and Homa Amini. 2017. “A Directed Batch Growing Approach to Enhance the Topology Preservation of Self-Organizing Map.” *Applied Soft Computing* 55 (June): 424–35. <https://doi.org/10.1016/j.asoc.2017.02.015>.
- Vettigli, Giuseppe. 2018. *JustGlowing/Minisom*.
- Wetmore, Leigh, Malcolm I. Heywood, and A. Nur Zincir-Heywood. 2005. “Speeding up the Self-Organizing Feature Map Using Dynamic Subset Selection.” *Neural Processing Letters* 22 (1): 17–32. <https://doi.org/10.1007/s11063-004-7775-6>.
- White, Denis, and A. Ross Kiester. 2008. “Topology Matters: Network Topology Affects Outcomes from Community Ecology Neutral Models.” *Computers, Environment and Urban Systems* 32 (2): 165–71. <https://doi.org/10.1016/j.compenvurbsys.2007.11.002>.
- Wittek, Peter, Shi Chao Gao, Ik Soo Lim, and Li Zhao. 2017. “Somoclu: An Efficient Parallel Library for Self-Organizing Maps.” *Journal of Statistical Software* 78 (June): 1–21. <https://doi.org/10.18637/jss.v078.i09>.

9. Supplementary Tables (End Matter)

Supplementary Table S1. Dataset metadata and numbered point key for the Figure 3 sampling-mode analysis.

dataset_index	dataset	dimension_count	sample_size	dataset_type
1	iris	4	150	real
2	wine	13	178	real
3	olivetti_faces	4096	400	real
4	diabetes	10	442	real
5	breast_cancer	30	569	real
6	digits	64	1797	real
7	california_housing	8	20640	real
8	blobs	2	30000	synthetic
9	circles	2	30000	synthetic
10	moons	2	30000	synthetic
11	s_curve	3	30000	synthetic
12	swiss_roll	3	30000	synthetic
13	kddcup99	41	494021	real
14	covertype	54	581012	real

Supplementary Table S2. HDSSSOM pilot configuration summary for Figure 2.

field	value
pilot purpose	Initial HDSSSOM screening before the broader sampling comparison
run envelope	Restricted hexagonal Optuna pilot
datasets	swiss_roll, moons, circles, blobs, s_curve, breast_cancer, wine, iris, digits, olivetti_faces
dataset count	10
seed count	5
sampling methods present	full, random, hdsssom
topology	hexagonal
algorithm families in campaign	batch, colors
optimization split setup	evaluation-split=both, yielding QE_H and QE_T ; Figure 2 reports paired QE_B
trials per scenario	200
main-text figure slice	Hexagonal / full_batch / full vs hdsssom

Supplementary Table S3. FloatSOM-versus-XPySOM calibration QE summary for the MST topology path. The dataset_index column matches the numbered points in Supplementary Figure S1 panel D. See assets/tables/supp_xpysom_calibration_qe_mst.tsv.

Supplementary Table S4. FloatSOM-versus-XPySOM calibration QE summary for the RNG topology path. The dataset_index column matches the numbered points in Supplementary Figure S2 panel D. See assets/tables/supp_xpysom_calibration_qe_rng.tsv.

Supplementary Table S5. FloatSOM-versus-XPySOM calibration QE summary for the hexagonal topology path. The dataset_index column matches the numbered points in Supplementary Figure S3 panel D. See assets/tables/supp_xpysom_calibration_QE_Hexagonal.tsv.

Supplementary Table S6. Paired topology-comparison p-values for hexagonal versus MST and hexagonal versus RNG across Balanced QE , Holdout QE , and Train QE . Rows list metric/dataset entries, including the OVERALL row. The MST and RNG columns report p-values using the manuscript reporting convention. See assets/tables/supp_table_topology_hex_vs_mst_rng_pvalues.tsv.

Supplementary Table S7. Figure 12 deployment-comparison percent summary for tuned FloatSOM RNG versus default hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change and 95% confidence interval. See assets/tables/supp_table_figure_11_xpysom_rng_deployment_summary.tsv.

Supplementary Table S8. Supplementary Figure S12 deployment-comparison percent summary for tuned FloatSOM hexagonal versus default hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change and 95% confidence interval. See assets/tables/supp_table_s13_xpysom_hexagonal_deployment_summary.tsv.

Supplementary Table S9. Supplementary Figure S13 deployment-comparison percent summary for tuned FloatSOM MST versus default hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change

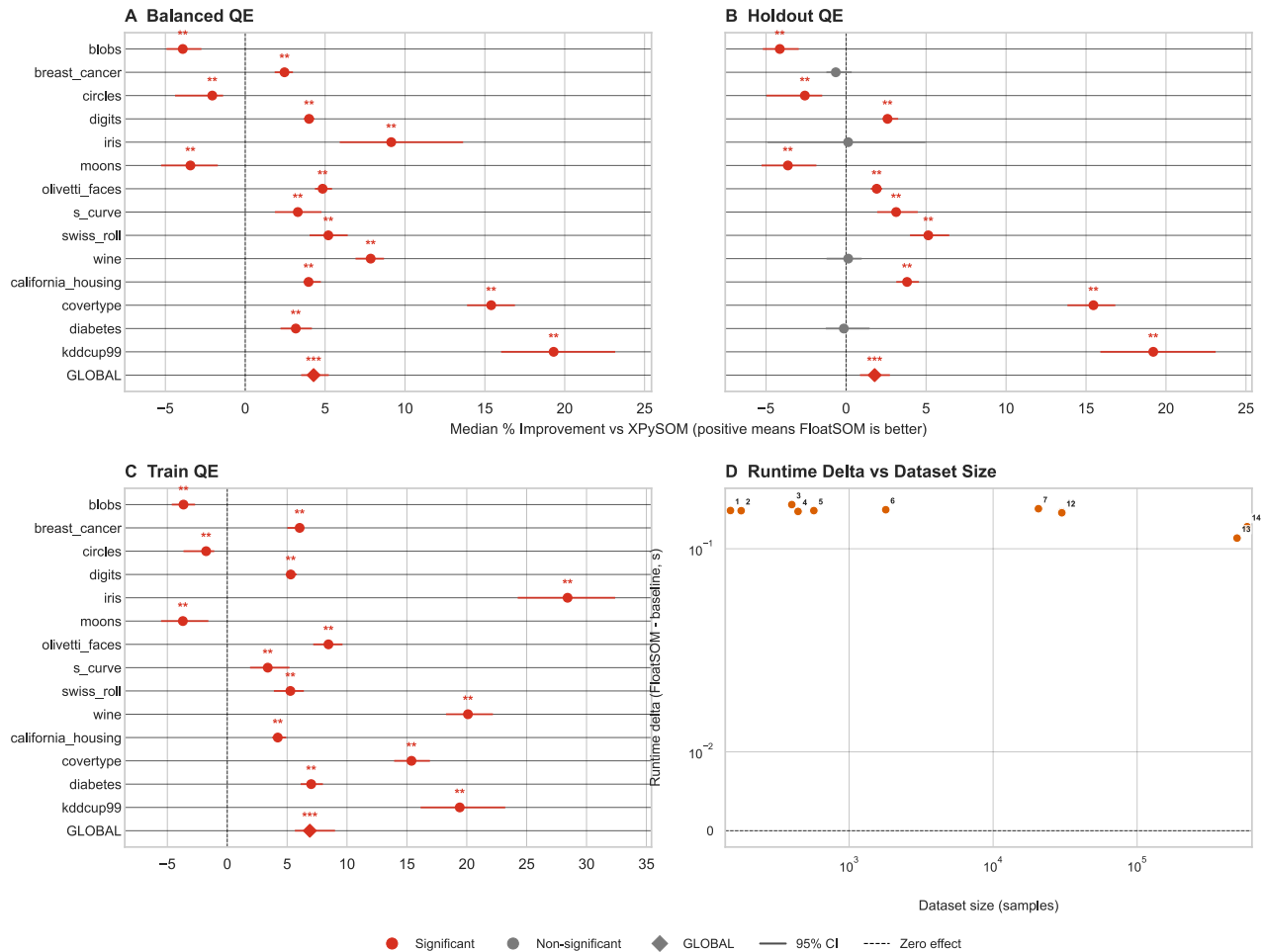
and 95% confidence interval. See

assets/tables/supp_table_s14_xpysom_mst_deployment_summary.tsv.

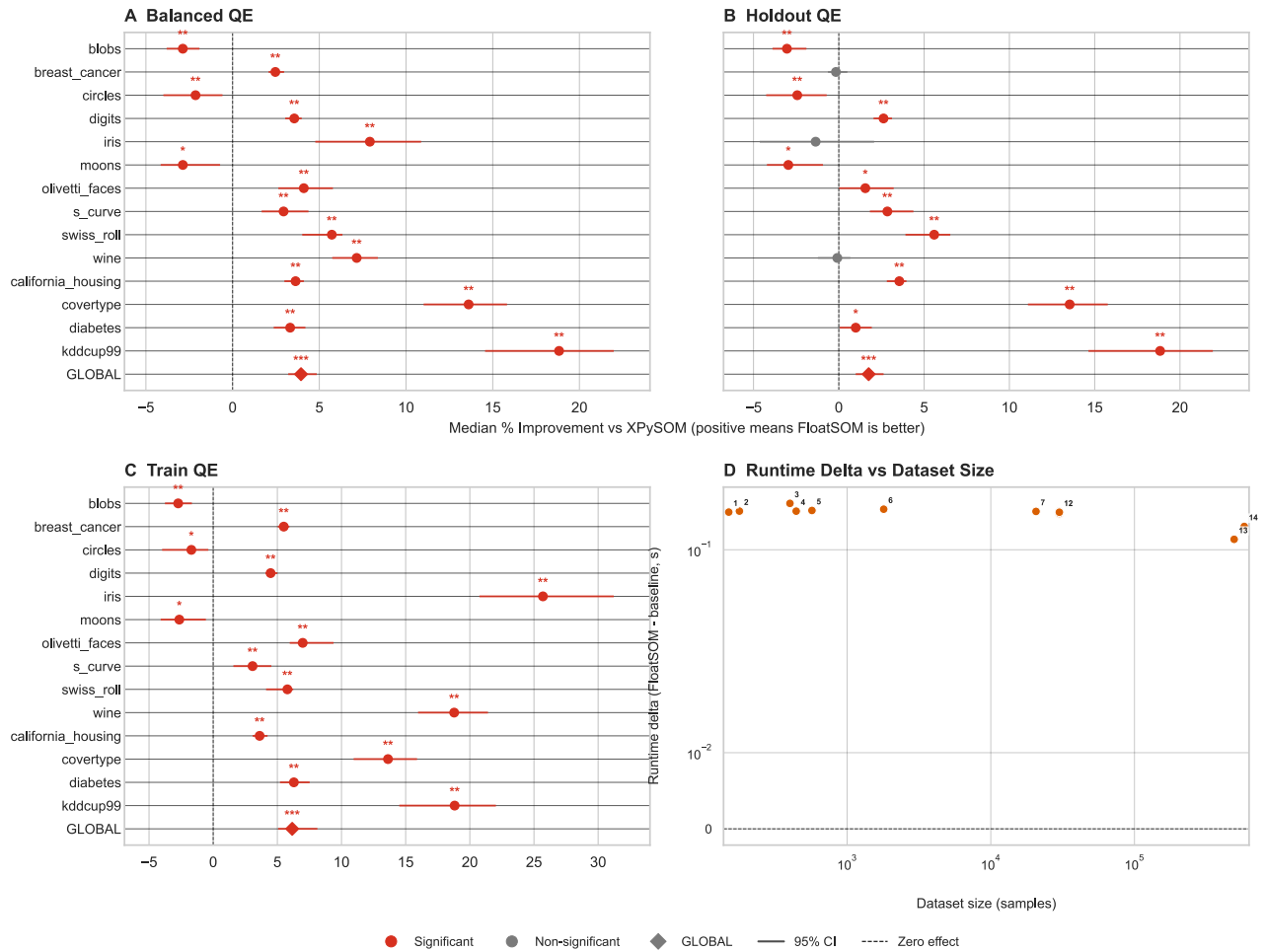
Supplementary Table S10. Figure 11 topology runtime summary at the largest common 8-GPU axis value for the dimension-, sample-, and grid-size-scaling workloads. Rows report the plotted 8-GPU mean runtimes for hexagonal, MST, and RNG, together with the fastest and slowest topology at that axis value and the maximum pairwise runtime spread. See assets/tables/supp_table_figure_10_topology_runtime_summary.tsv.

10. Supplementary Figures (End Matter)

Supplementary Figure S1: FloatSOM vs XPySOM Calibration (Default Settings, MST)

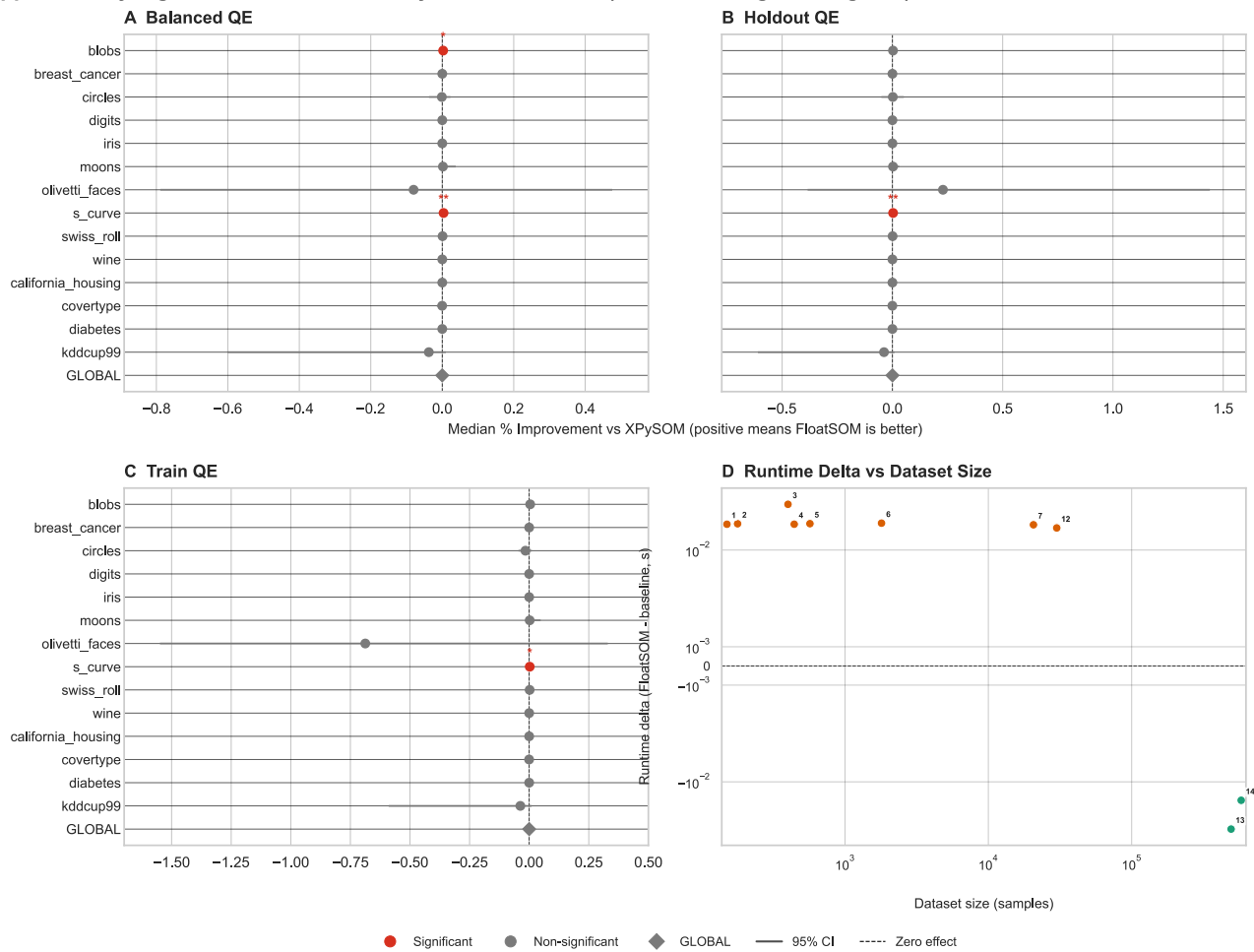


Supplementary Figure S1. FloatSOM-versus-XPySOM calibration under default settings for the MST topology path. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of FloatSOM time - XPySOM time; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S3. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S2: FloatSOM vs XPySOM Calibration (Default Settings, RNG)

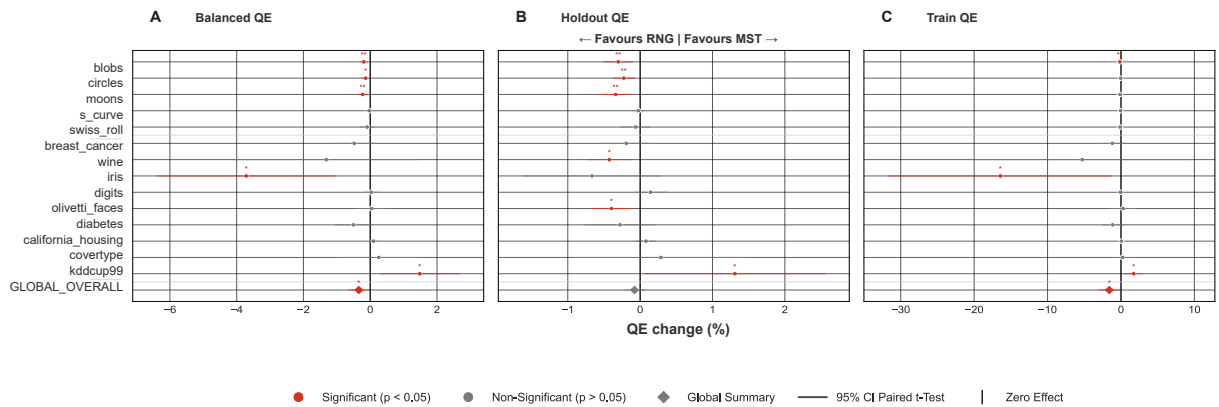
Supplementary Figure S2. FloatSOM-versus-XPySOM calibration under default settings for the RNG topology path. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of FloatSOM time - XPySOM time; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S4. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S3: FloatSOM vs XPySOM Calibration (Default Settings, Hexagonal)



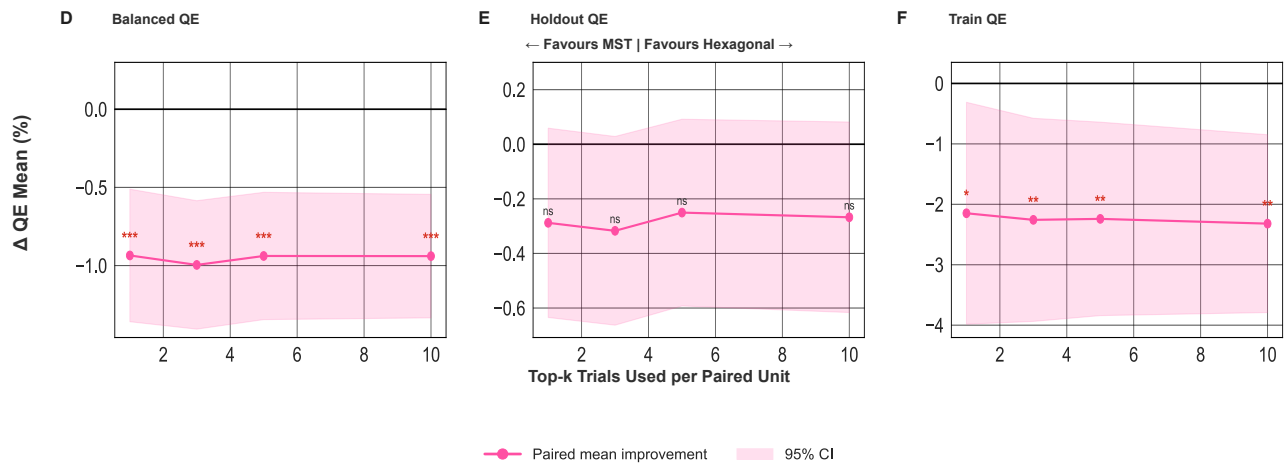
Supplementary Figure S3. FloatSOM-versus-XPySOM calibration under default settings for the hexagonal topology path. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of FloatSOM time - XPySOM time; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S5. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S5: MST vs RNG Across QE Metrics



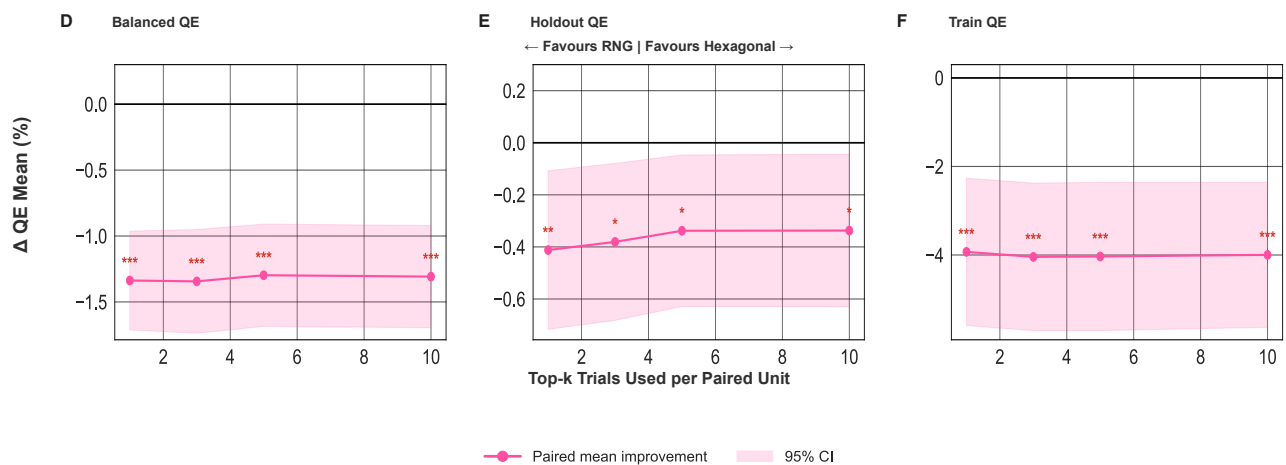
Supplementary Figure S4. MST versus RNG topology on QE endpoints under full sampling only. Panels A-C report paired full-sampling-only QE effects for QE_B , QE_H , and QE_T across the available full-sampling datasets. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S6: Hexagonal vs MST Sensitivity Across QE Metrics



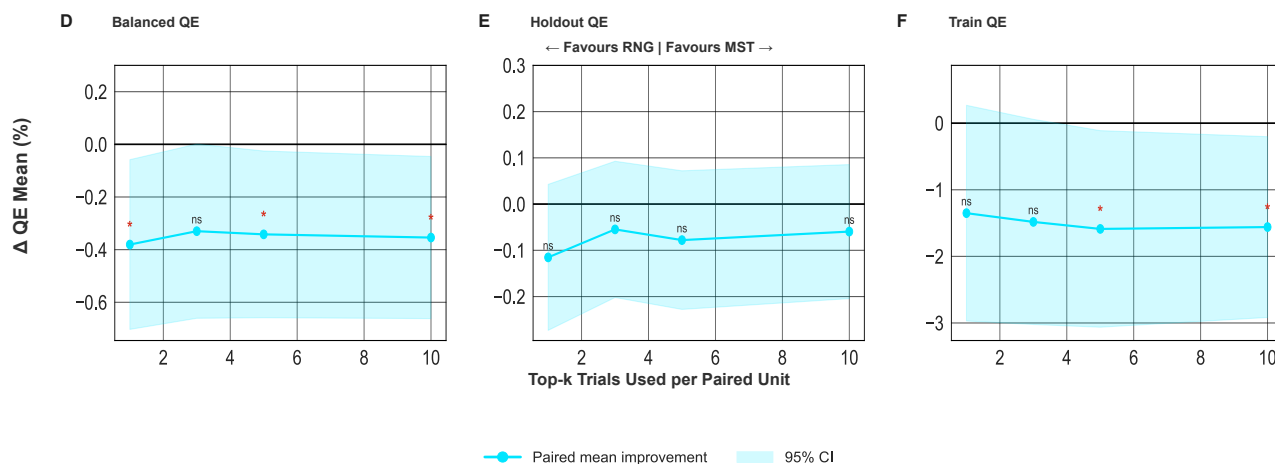
Supplementary Figure S5. Hexagonal versus MST topology sensitivity under full sampling only. Panels A-C report the matched top-k paired sensitivity analysis for QE_B , QE_H , and QE_T .

Supplementary Figure S7: Hexagonal vs RNG Sensitivity Across QE Metrics



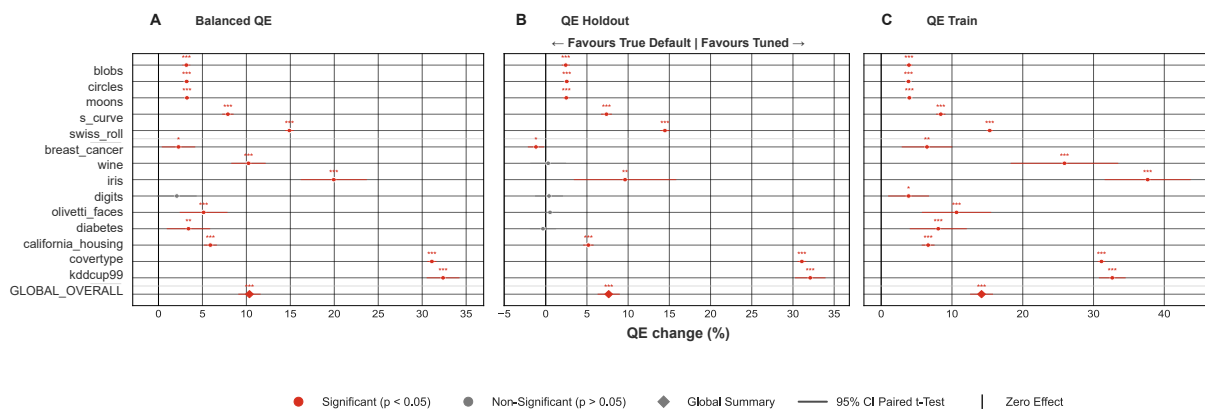
Supplementary Figure S6. Hexagonal versus RNG topology sensitivity under full sampling only. Panels A-C report the matched top-k paired sensitivity analysis for QE_B , QE_H , and QE_T .

Supplementary Figure S8: MST vs RNG Sensitivity Across QE Metrics



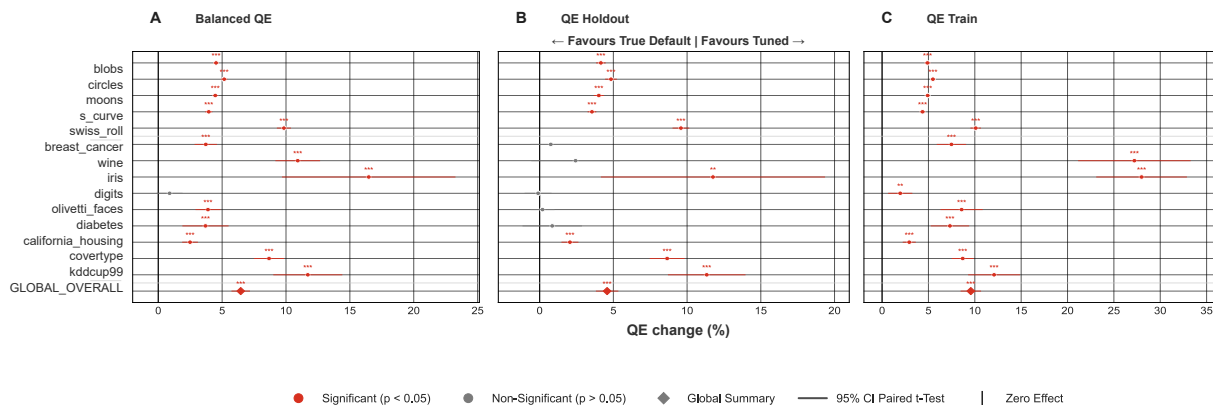
Supplementary Figure S7. MST versus RNG topology sensitivity under full sampling only. Panels A-C report the matched top-k paired sensitivity analysis for QE_B , QE_H , and QE_T .

Supplementary Figure S9: Tuned vs True Default Across QE Metrics (Hexagonal)



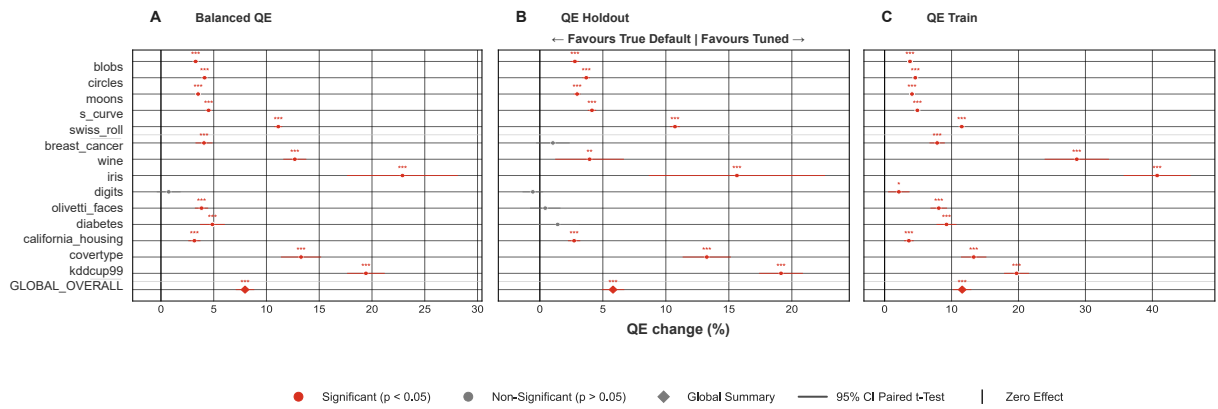
Supplementary Figure S8. Tuned-configuration-versus-untuned-reference QE comparison for the hexagonal topology only, across QE_B , QE_H , and QE_T under the matched pairing keys. Positive values indicate the tuned configuration outperforms the untuned reference. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S10: Tuned vs True Default Across QE Metrics (MST)



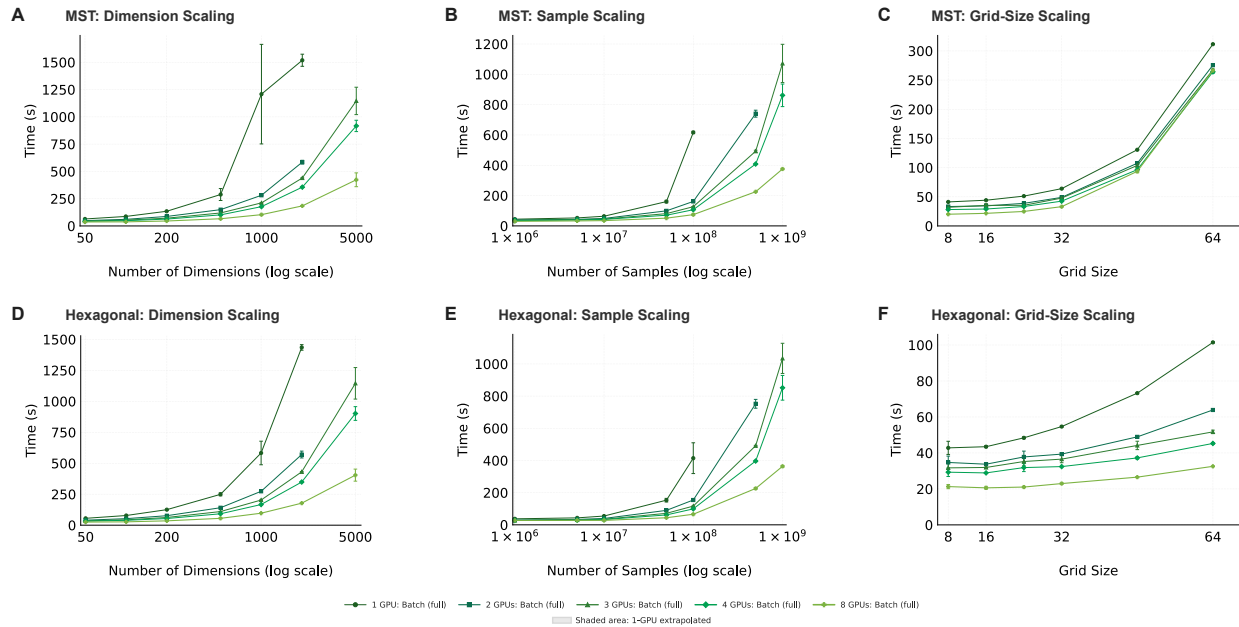
Supplementary Figure S9. Tuned-configuration-versus-untuned-reference QE comparison for the MST topology only, across QE_B , QE_H , and QE_T under the matched pairing keys. The tuned configuration is derived from the Optuna-selected settings by taking the mean of numeric parameters and the mode of categorical parameters across seeds. Positive values indicate the tuned configuration outperforms the untuned reference. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S11: Tuned vs True Default Across QE Metrics (RNG)



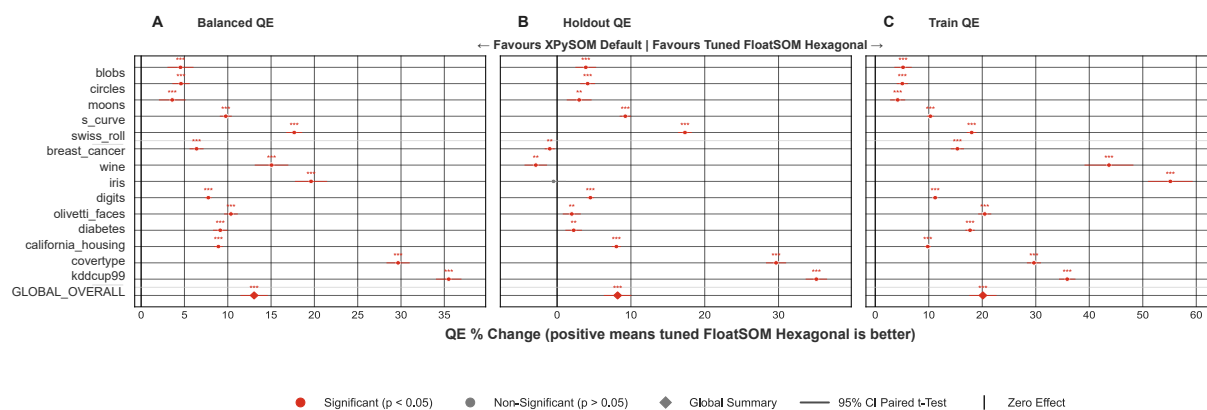
Supplementary Figure S10. Tuned-configuration-versus-untuned-reference QE comparison for the RNG topology only, across QE_B , QE_H , and QE_T under the matched pairing keys. The tuned configuration is derived from the Optuna-selected settings by taking the mean of numeric parameters and the mode of categorical parameters across seeds. Positive values indicate the tuned configuration outperforms the untuned reference. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect.

Supplementary Figure S12: Per-Topology GPU Scaling Performance (MST and Hexagonal)



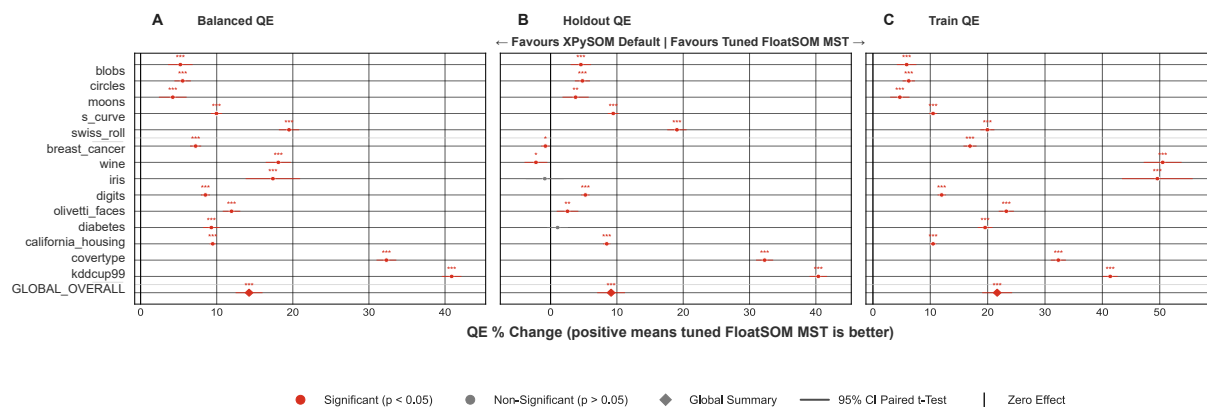
Supplementary Figure S11. Full GPU-count scaling context for MST and Hexagonal under matched full-batch settings. Panels A-C show MST runtime across dimension-, sample-, and grid-size-scaling workloads; panels D-F show the corresponding Hexagonal runs. Within each panel, curves correspond to $G \in \{1, 2, 4, 8\}$ GPUs and report mean wall-clock runtime (s) with ± 1 standard-deviation error bars across $n = 3$ repeated runs per configuration.

Supplementary Figure S13: XPySOM (Default) v FloatSOM Hexagonal



Supplementary Figure S12. Deployment comparison of default hexagonal XPySOM versus tuned FloatSOM hexagonal. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T under the matched dataset/seed comparison keys. Positive values indicate tuned FloatSOM hexagonal outperforms default hexagonal XPySOM. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect. Per-dataset and GLOBAL_OVERALL panel summaries are listed in Supplementary Table S8.

Supplementary Figure S14: XPySOM (Default) v FloatSOM MST



Supplementary Figure S13. Deployment comparison of default hexagonal XPySOM versus tuned FloatSOM MST. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T under the matched dataset/seed comparison keys. Positive values indicate tuned FloatSOM MST outperforms default hexagonal XPySOM. Forest whiskers denote 95% paired t-test confidence intervals around the mean paired effect. Per-dataset and GLOBAL_OVERALL panel summaries are listed in Supplementary Table S9.